

Resumen de Teoría de:

Diseño de Sistemas

Autor: Adrián Botta

Año: 2010

Fuentes:

UML: proceso unificado y Manual de referencia – Rumbaugh, Jacobson, Booch
UML y patrones – Larman
Apuntes de Cátedra
RUP - A.U.S. Gustavo Torossi
Wikipedia

**Resumen Realizado en base al Programa 2010*

PROGRAMA 2010	
UNIDAD	CONTENIDO
1	<u>Introducción al Diseño</u> 1.1. ¿Qué es el diseño? Conceptos-Características 1.2. Diferencia entre análisis y diseño 1.3. Objetivos y Alcances 1.4. Límites de Automatización 1.5. Revisión de los principios de Diseño Orientado a Objetos
2	<u>El Proceso de Desarrollo Unificado</u> 2.1. ¿Qué es el Proceso unificado? Conceptos-Características 2.2. Conceptos Básicos del Proceso: Flujos, artefactos, trabajadores, actividades 2.3. Tipos de Flujos: Centrales y de Iteración 2.4. Flujo de Trabajo de Captura de Requerimientos 2.5. Modelo de Casos de uso 2.6. Modelo del Dominio 2.6.1. Visualización de Conceptos 2.6.2. Identificación de clases conceptuales 2.6.3. Relaciones y Atributos 2.7. Prototipo de interfaz de usuario 2.8. Flujo de Trabajo de Análisis 2.9. Artefactos del modelo de análisis 2.10. Modelo de Análisis
3	<u>Flujo de Trabajo de Diseño</u> 3.1. Workflow de Diseño: Consideraciones 3.2. El rol del diseño en el ciclo de vida del software 3.3. Artefactos de Diseño: Subsistema de diseño, Interfaz, Descripción Arquitectura (VMDi), Modelo de Despliegue, Descripción Arquitectura (VMDe) 3.4. Trabajadores del diseño 3.5. Actividades del Diseño 3.6. Frameworks 3.7. Patrones GRASP: Experto, Creador, Bajo acoplamiento, Alta Cohesión, Controlador, Polimorfismo, Indirección, Fabricación Pura, Variaciones Protegidas 3.8. Patrones GoF: Estrategia, Adaptador, Observador, Composite, DTO 3.9. Modelo Entidad Relación
4	<u>Flujo de Trabajo de Implementación y Pruebas</u> 3.1. Workflow de Implementación: Consideraciones 3.2. El rol de la Implementación en el ciclo de vida del software 3.3. Artefactos, Trabajadores y Actividades de la Implementación 3.4. Workflow de Pruebas: Consideraciones 3.5. El rol de la Prueba en el ciclo de vida del software 3.6. Artefactos, Trabajadores y Actividades de la Prueba

ÍNDICE DEL RESUMEN

Unidad 1: El Proceso de Desarrollo Unificado

1. ¿Qué es el Proceso unificado? Conceptos-Características
2. Conceptos Básicos del Proceso: Flujos, artefactos, trabajadores, actividades

Unidad 2: Flujos de Trabajo Fundamentales

1. Flujo de Trabajo de Captura de Requisitos
 - a. Modelo de Casos de uso
 - b. Modelo del Dominio: Visualización de conceptos y clases conceptuales
 - c. Relaciones y Atributos
 - d. Prototipo de interfaz de usuario
2. Flujo de Trabajo de Análisis
 - a. El rol del Análisis en el ciclo de vida del software
 - b. Artefactos del modelo de análisis: Modelo de Análisis
 - c. Trabajadores del Análisis
 - d. Actividades del Análisis
3. Flujo de Trabajo de Diseño
 - a. ¿Qué es el diseño? Conceptos-Características
 - b. El rol del diseño en el ciclo de vida del software
 - c. Diferencia entre análisis y diseño
 - d. Objetivos y Alcances
 - e. Límites de Automatización
 - f. Artefactos de Diseño: Subsistema de diseño, Interfaz, Descripción Arquitectura (VMDi), Modelo de Despliegue, Descripción Arquitectura (VMDe)
 - g. Trabajadores del diseño
 - h. Actividades del Diseño
4. Flujo de Trabajo de Implementación
 - a. El rol de la Implementación en el ciclo de vida del software
 - b. Artefactos del modelo de Implementación
 - c. Trabajadores de la Implementación
 - d. Actividades de la Implementación
5. Flujo de Trabajo de Pruebas
 - a. El rol de las Pruebas en el ciclo de vida del software
 - b. Tipos de Pruebas
 - c. Artefactos del modelo de Pruebas
 - d. Trabajadores de las Pruebas
 - e. Actividades de las Pruebas

Unidad 3: Patrones

1. Patrones GRASP: Experto, Creador, Bajo acoplamiento, Alta Cohesión, Controlador, Polimorfismo, Indirección, Fabricación Pura, Variaciones Protegidas
2. Patrones GoF: Estrategia, Adaptador, Observador, Composite, DTO, Plantilla, Singleton, etc.

Unidad 4: Diseño Orientado a Objetos

1. Revisión de los principios de Diseño Orientado a Objetos
2. Modelo Entidad Relación
3. Frameworks

Unidad 5: UML

1.1 ¿Qué es el Proceso unificado? Conceptos-Características

El proceso unificado (PU) es un proceso de desarrollo de software. Un proceso de desarrollo de software es un conjunto de actividades necesarias para transformar los requisitos de un usuario en un sistema software. El proceso unificado es un marco de trabajo genérico que puede especializarse para una gran variedad de sistemas de software, para diferentes áreas de aplicación, diferentes tipos de organizaciones, diferentes niveles de aptitud y diferentes tamaños de proyecto. Este proceso está basado en componentes interconectados a través de interfaces bien definidas.

El Proceso unificado utiliza el Lenguaje de Modelado Unificado (UML) para preparar todos los esquemas de un sistema de software.

Los aspectos definitorios del PU son:

- **Dirigido por casos de usos:** Un sistema software dará servicio a sus usuarios, por tanto, debemos conocer lo que sus futuros usuarios necesitan y desean, ya que el usuario interactuará con el sistema. Una interacción de este tipo es un caso de uso. Un caso de uso es un fragmento de funcionalidad del sistema que proporciona al usuario un resultado importante. Es decir, los CU representan los requisitos funcionales, y entre todos constituyen el modelo de casos de uso, el cual describe la funcionalidad total del sistema. Debemos preguntarnos ¿Qué debe hacer el sistema....para cada usuario?, por eso los casos de uso guían el proceso de desarrollo, proporcionan un hilo conductor para el proceso.

Aunque los casos de uso guían el proceso, se desarrollan conjuntamente con la arquitectura del sistema, y ambos maduran según avanza el ciclo de desarrollo.

- **Centrado en la arquitectura:** El concepto de arquitectura incluye los aspectos estáticos y dinámicos más significativos del sistema. La arquitectura surge de las necesidades de la empresa, como la perciben los usuarios y los inversores, y se refleja en los casos de uso. La arquitectura es una vista del diseño completo con las características más importantes resaltadas, dejando los detalles de lado. Los CU deben encajar en la arquitectura cuando se llevan a cabo, y la arquitectura debe permitir el desarrollo de todos los casos de uso requeridos, ahora y en el futuro. La arquitectura debe diseñarse para permitir que el sistema evolucione en su desarrollo inicial y a lo largo de las futuras generaciones. Para encontrar esa forma, los arquitectos deben trabajar sobre la comprensión general de las funciones clave, es decir, sobre los casos de uso claves del sistema.
- **Iterativo e Incremental:** Los desarrolladores basan la selección de lo que se implementará en una iteración en dos factores. En 1º lugar, la iteración trata un grupo de casos de uso que juntos amplían la utilidad del producto desarrollado hasta ahora. En 2º lugar, la iteración trata los riesgos más importantes. Las iteraciones sucesivas se construyen sobre los artefactos de desarrollo tal como quedaron al final de la última iteración. En cada iteración, los desarrolladores identifican y especifican los casos de uso relevantes, crean un diseño utilizando la arquitectura seleccionada como guía, implementan el diseño mediante componentes, y verifican que los componentes satisfacen los casos de uso. Si una iteración cumple con sus objetivos, el desarrollo continúa con la siguiente iteración. Si no, se prueba con un nuevo enfoque.

El Proceso unificado se repite a lo largo de una serie de ciclos que constituyen la vida de un sistema. Cada ciclo consta de 4 fases, (cada una de las cuales se divide en iteraciones):

- 1- Fase de inicio:** Se desarrolla una descripción del producto final a partir de una buena idea y se presenta el análisis de negocio para el producto
- 2- Fase de Elaboración:** Se especifican en detalle la mayoría de los casos de uso del producto y se diseña la arquitectura del sistema. El resultado de esta fase es una línea base de la arquitectura, y la disposición del director de planificar las actividades y estimar los recursos necesarios para terminar el proyecto
- 3- Fase de Construcción:** Se crea el producto. Aquí la línea base de la arquitectura crece hasta convertirse en el sistema completo. Al final de esta fase, el producto tiene todos los casos de uso que la dirección y el cliente han acordado para el desarrollo de esta versión. Sin embargo, puede tener defectos
- 4- Fase de Transición:** Cubre el periodo comprendido durante el cual el producto se convierte en versión beta. Esta fase corrige errores antes de la entrega. El equipo de mantenimiento divide los errores en 2 categorías: los que tienen suficiente impacto en la operación para justificar una versión incrementada y los que pueden corregirse en la siguiente versión normal.

Cada ciclo produce una nueva versión del sistema, y cada versión es un producto preparado para su entrega. El producto terminado incluye los requisitos, casos de uso, especificaciones no funcionales y casos de prueba. Incluye el modelo de la arquitectura y el modelo visual.

1.2 Conceptos Básicos del Proceso: Flujos, artefactos, trabajadores, actividades

Trabajadores (“quién”): Define el comportamiento y responsabilidades (rol) de un individuo, grupo de individuos, sistema automatizado o máquina, que trabajan en conjunto como un equipo. Ellos realizan las actividades y son propietarios de elementos.

Actividades (“cómo”): Es una tarea que tiene un propósito claro, es realizada por un trabajador, que manipula elementos. Las actividades se consideran en la planificación y evaluación de progresos del proyecto

Artefactos (“qué”): Productos tangibles del proyecto que son producidos, modificados y usados por las actividades. Pueden ser modelos, elementos dentro del modelo, código fuente y ejecutables.

Flujo de actividades (“Cuándo”): Secuencia de actividades realizadas por trabajadores y que produce un resultado de valor observable. Los flujos de trabajo principales son:

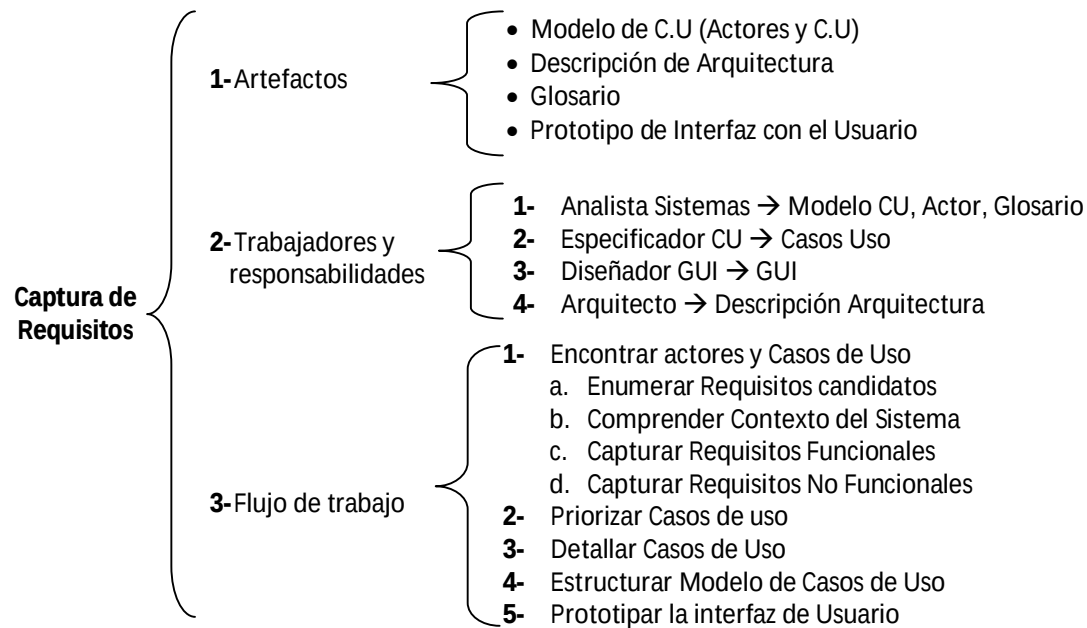
- Captura de Requerimientos: Define qué es lo que el sistema debe hacer, para lo cual se identifican las funcionalidades requeridas y las restricciones que se imponen.
- Análisis y diseño: Describe cómo el sistema será realizado a partir de la funcionalidad prevista y las restricciones impuestas (requerimientos), por lo que indica con precisión lo que se debe programar.
- Implementación: Define cómo se organizan las clases y objetos en componentes, cuáles nodos se utilizarán y la ubicación en ellos de los componentes y la estructura de capas de la aplicación.
- Prueba (Testeo): Busca los defectos a lo largo del ciclo de vida.

UNIDAD 2: FLUJOS DE TRABAJO FUNDAMENTALES

	Actividad	Trabajador	Artefacto Entrada	Artefacto Salida
Requisitos	1 Encontrar Actores y CU	Analista en Sistemas	+ Modelo Negocio o Dominio + Requisitos Adicionales + Lista de Características	+ Modelo CU (esbozado) + Glosario
	2 Priorizar CU	Arquitecto	+ Modelo CU (esbozado) + Glosario	Descripción de la Arquitectura (Vista del MCU)
	3 Detallar CU	Especificador de CU	+ Requisitos Adicionales	+ CU Detallado
	4 Estructurar Modelo de CU	Analista en Sistemas	+ Modelo CU (esbozado) + Glosario + Requisitos Adicionales + CU Detallado	+ Modelo CU (terminado)
	5 Prototipar IU	Diseñador de IU	+ Modelo CU + Glosario + Requisitos Adicionales + CU Detallado	Prototipo de IU
Análisis	1 Análisis de la Arquitectura	Arquitecto	+ Modelo Negocio o Dominio + Requisitos Adicionales + Modelo de CU + Descripción Arquitectura (VMCU)	+ Descripción Arquitectura (VMA) + Clase de Análisis (esbozada) + Paquete de Análisis (esbozado)
	2 Analizar un CU	Ing. En CU	+ Modelo Negocio o Dominio + Requisitos Adicionales + Modelo de CU + Descripción Arquitectura (VMA)	+ Realización de un CU (análisis) + Clase de Análisis (esbozada)
	3 Analizar una Clase	Ing. en Componentes	+ Realización de un CU (análisis) + Clase de Análisis (esbozada)	+ Clase de Análisis (terminado)
	4 Analizar un Paquete	Ing. en Componentes	+ Descripción Arquitectura (VMA) + Paquete de Análisis (esbozado)	+ Paquete de Análisis (terminado)
Diseño	1 Diseño de la Arquitectura	Arquitecto	+ Modelo de CU + Requisitos Adicionales + Modelo de Análisis + Descripción Arquitectura (VMA)	+ Descripción Arquitectura (VMD) + Modelo de Despliegue + Subsistema (esbozado) + Interfaz (esbozada) + Clase de diseño (esbozada)
	2 Diseño de un CU	Ing. En CU	+ Modelo de CU + Requisitos Adicionales + Modelo de Análisis + Modelo de Diseño + Modelo de Despliegue	+ Subsistema (esbozado) + Interfaz (esbozada) + Clase de diseño (esbozada) + Realización de un CU (diseño)
	3 Diseño de una Clase	Ing. En Componentes	+ Realización de un CU (diseño) + Interfaz (esbozada) + Clase de análisis (terminada) + Clase de diseño (esbozada)	+ Clase de diseño (terminada)
	4 Diseño de un Subsistema	Ing. En Componentes	+ Descripción Arquitectura (VMD) + Subsistema (esbozado) + Interfaz (esbozada)	+ Subsistema (terminado) + Interfaz (terminada)

Implementación	1 Impl. De la Arquitectura	Arquitecto	+ Descripción Arquitectura (VMD) + Modelo de Despliegue + Modelo de Diseño	+ Componente (esbozado y posiblemente asignado a nodos) + Descripción Arquitectura (VMI)
	2 Integrar el Sistema	Integrador de Sistemas	+ Requisitos Adicionales + Modelo de CU + Modelo de Diseño + Modelo de Implementación (construcciones anteriores)	+ Plan de Int. de Construcciones + Modelo de Implementación (construcciones anteriores)
	3 Impl. Un subsistema	Ing. En Componentes	+ Plan de Int. de Construcciones + Descripción Arquitectura (VMI) + Subsistema de Diseño (terminado) + Interfaz (terminada)	+ Subsistema de Implementación (impl. Para una construcción) + Interfaz (impl. Para una construcción)
	4 Implementar Una clase	Ing. En Componentes	+ Clase de diseño (terminada) + Interfaz (de la clase de diseño)	+ Componente (implementado)
	5 Realizar prueba de unidad	Ing. En Componentes	+ Componente (implementado) + Interfaz	+ Componente (probado)
Prueba	1 Planificar Prueba	Ing. De Pruebas	+ Requisitos Adicionales + Modelo de CU + Modelo de Diseño + Modelo de Implementación + Descripción de la Arquitectura (Vistas arquitect. de los modelos)	+ Plan de Prueba
	2 Diseñar Prueba	Ing. De Pruebas	+ Requisitos Adicionales + Modelo de CU + Modelo de Diseño + Modelo de Implementación + Descripción de la Arquitectura (Vistas arquitect. de los modelos) + Plan de Prueba	+ Caso de Prueba + Procedimiento de Prueba
	3 Implementar Prueba	Ing. En Componentes	+ Caso de Prueba + Procedimiento de Prueba + Modelo de Implementación (construida para ser probada)	+ Componente de Prueba
	4 Pruebas de Integración	Ing. Pruebas Integración	+ Caso de Prueba + Procedimiento de Prueba + Componente de Prueba + Modelo de Implementación (construcción a probar)	+ Defecto
	5 Pruebas de Sistema	Ing. Pruebas de Sistema		
	6 Evaluar Prueba	Ing. De Pruebas	+ Plan de Prueba + Modelo de Prueba + Defecto	+ Evaluación de prueba (para una iteración)

2.1 FLUJO DE TRABAJO DE CAPTURA DE REQUISITOS



Disciplina de Requisitos: Ejecuta la identificación y captura de las necesidades que el proyecto debe cubrir con lo desarrollado.

El propósito fundamental del flujo de trabajo de los requisitos es guiar el desarrollo hacia el sistema correcto. Esto se consigue mediante una descripción de los requisitos del sistema suficientemente buena como para que pueda llegarse a un acuerdo entre el cliente y los desarrolladores sobre qué debe y qué no debe hacer el sistema.

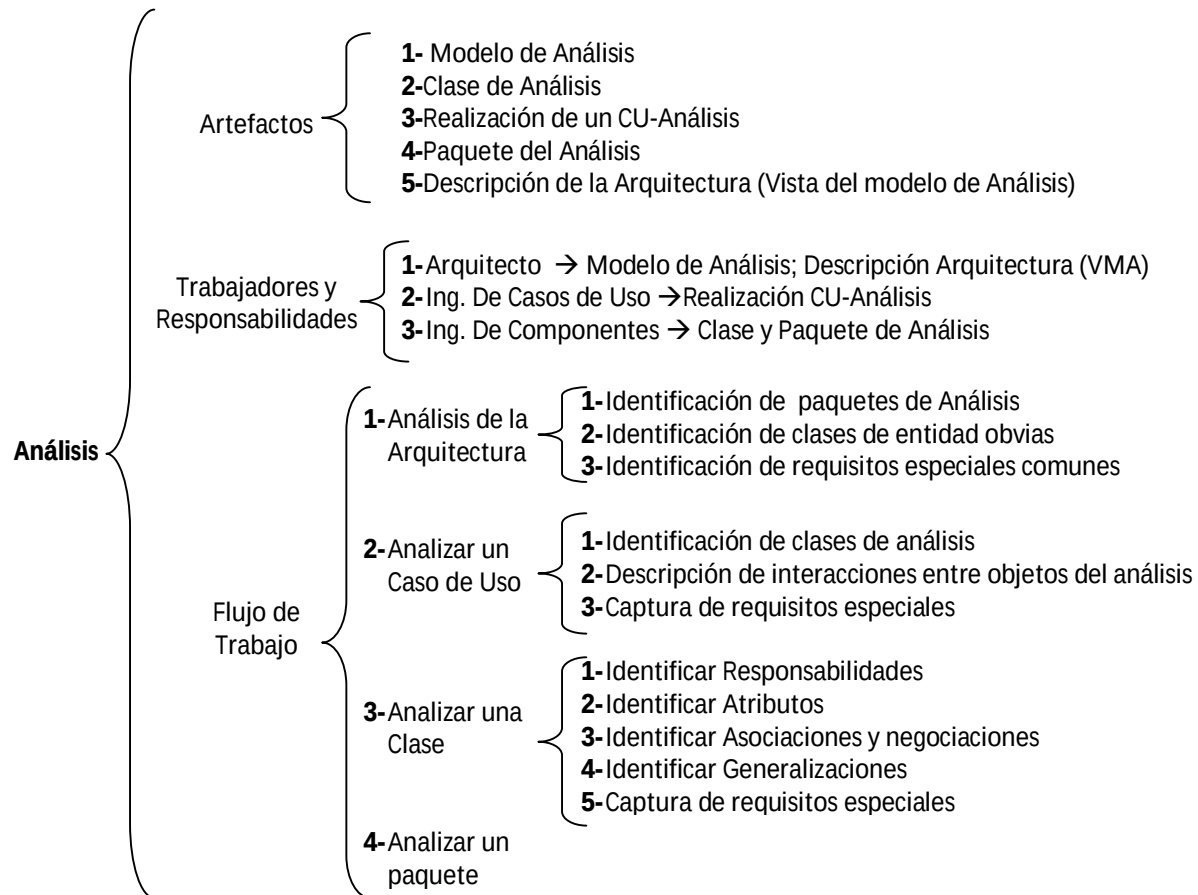
Artefactos de la Captura de Requisitos

- Actor: Es toda entidad externa que demanda funcionalidad del sistema, ya sea un ser humano o un sistema de software. Los actores nos sirven para capturar a los usuarios del sistema pero también, son la forma en la que podemos expresar el contexto del mismo. Es decir, que los actores toman el lugar de cualquier entidad de interés con la que nuestro sistema interactúa.
- Caso de uso: Especifica una secuencia de acciones, incluyendo variantes, que el sistema puede llevar a cabo, y que producen un resultado observable de valor para un actor concreto.
- Modelo de CU: Describe los procesos de negocio de una empresa en términos de casos de uso y actores del negocio que se corresponden con los procesos del negocio y los clientes, respectivamente. El modelo de casos de uso del negocio presenta un sistema desde la perspectiva de su uso, y esquematiza cómo proporciona valor a sus usuarios. Este modelo se describe mediante diagramas de casos de uso.
- Descripción de la Arquitectura
- Prototipo de IU
- Glosario: Diccionario de Términos

Flujo de Trabajo de la Captura de Requisitos

- 1- **Encontrar Actores y Casos de uso:** Identificamos los actores y casos de uso para delimitar el sistema de su entorno, y esbozar quién (actores) interactuarán con el sistema, y qué funcionalidad (casos de uso) se espera del sistema. Comprende las sub-etapas siguientes:
 - a. Enumerar los requisitos candidatos: Es la lista de características deseadas del sistema
 - b. Comprender el contexto del sistema: Para capturar los requisitos correctos y para construir el sistema correcto, los desarrolladores clave requieren un firme conocimiento del contexto en el que se emplaza el sistema. Hay 2 formas para esto:
 - i. *Modelo de dominio:* se aplica cuando el analista es experto en ese tipo de sistema, con lo que evita gran parte de la captura de requisitos, realizando afirmaciones sobre determinados aspectos del sistema. Un modelo de dominio captura los tipos más importantes de objetos en el contexto del sistema. Los objetos de dominio representan las “cosas” que existen o los eventos que suceden en el entorno en el que trabaja el sistema. Las clases del dominio aparecen en tres formas típicas:
 - o Objetos del negocio que representan cosas que se manipulan en el negocio
 - o Objetos del mundo real y conceptos de los que el sistema debe conocer
 - o Sucesos que ocurrirán o han ocurrido
 - ii. *Modelo de negocio:* es más amplio que el modelo del dominio. Describe los procesos con el objetivo de comprenderlos. El modelado del negocio especifica que procesos de negocio soportará el sistema. Un modelo de objetos del negocio describe como cada caso de uso del negocio es llevado a cabo por parte de un conjunto de trabajadores que utilizan un conjunto de entidades del negocio y de unidades de trabajo. Una **entidad** del negocio representa algo que los trabajadores toman, manipulan, inspeccionan, producen o utilizan en un negocio.
 - c. Capturar requisitos funcionales: Un requisito funcional es una característica requerida del sistema que expresa una capacidad de acción del mismo (una funcionalidad); generalmente expresada en una declaración en forma verbal. Se traducen directamente en casos de uso
 - d. Capturar requisitos no funcionales: Especifican propiedades del sistema, como restricciones del entorno o de la implementación, rendimiento, dependencias de la plataforma, facilidad de mantenimiento, extensibilidad y fiabilidad. Los requisitos no funcionales más genéricos se denominan requisitos adicionales.
- 2- **Priorizar casos de uso:** El propósito de esta actividad es determinar cuáles casos de uso son necesarios para el desarrollo en las primeras iteraciones, y cuáles pueden dejarse para más tarde.
- 3- **Detallar un caso de uso:** Consiste en describir su flujo de sucesos en detalle, incluyendo cómo comienza el CU, termina e interactúa con los actores. El especificador detalla paso a paso la descripción de cada caso de uso en una especificación precisa de la secuencia de acciones.
- 4- **Estructurar el Modelo de Casos de Uso**
- 5- **Prototipar la interfaz de usuario:** Consiste en construir un prototipo de interfaz de usuario. Necesitamos diseñar la interfaz de usuario que permita al usuario llevar a cabo los casos de uso de manera eficiente.

2.2 FLUJO DE TRABAJO DE ANÁLISIS



Análisis: Actividad mental de ver al todo como una colección de partes.

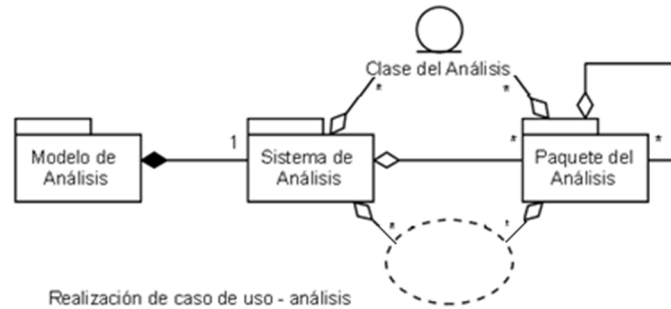
Disciplina de Análisis: Ordena en un esquema lógico lo que se ha implicado con los requisitos.

Durante el análisis, analizamos los requisitos que se describieron en la captura de requisitos, refinándolos y estructurándolos. El objetivo de hacerlo es conseguir una comprensión más precisa de los requisitos y una descripción de los mismos que sea fácil de mantener, y que nos ayude a estructurar el sistema entero.

Modelo de Casos de Uso	Modelo de Análisis
Descrito con el lenguaje del cliente	Descrito con el lenguaje del desarrollador
Vista externa del sistema	Vista interna del sistema
Estructurado por los casos de uso	Estructurado por clases y paquetes
Utilizado como contrato entre el cliente y desarrolladores	Utilizado por los desarrolladores para comprender como se debería diseñar e implementar el sistema
Puede contener redundancias, inconsistencias, etc., entre requisitos	No debería contener redundancias, inconsistencias, etc., entre requisitos
Captura la funcionalidad del sistema, incluida la funcionalidad significativa para la arquitectura	Esboza cómo llevar a cabo la funcionalidad dentro del sistema, incluida la funcionalidad significativa para la arquitectura. Sirve como una 1ª aproximación al diseño.
Define casos de uso que se analizarán con más profundidad en el modelo de análisis	Define realizaciones de casos de uso, y cada una de ellas representa el análisis de un caso de uso del modelo de casos de uso

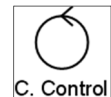
Artefactos del Análisis

- Modelo de Análisis



- Clase del análisis:** Representa una abstracción de una o varias clases y/o subsistemas del diseño del sistema. Este tipo de clase se centra en el tratamiento de los requisitos funcionales y pospone los no funcionales (especiales). Además define atributos, pero con gran abstracción. Participa en relaciones del tipo conceptual, es decir, son distintas a las relaciones de implementación. Encajan en uno de los 3 estereotipos siguientes:
 - Clase de Interfaz:** Se utilizan para modelar la interacción entre el sistema y sus actores. Esta interacción a menudo implica recibir (y presentar) información y peticiones de (y hacia) los usuarios y los sistemas externos. Las clases de interfaz representan a menudo abstracciones de ventanas, formularios, sensores, API's, etc. Cada clase de interfaz debería asociarse con un actor y viceversa.
 
 - Clase de Entidad:** Usadas para modelar información de vida larga y que es a menudo persistente. Reflejan la información de un modo que beneficia a los desarrolladores al diseñar e implementar el sistema, incluyendo persistencia
 
 - Clase de Control:** Representan coordinación, secuencia, transacciones y control de otros objetos. Se usan para encapsular el control de un CU en concreto, y para representar derivaciones y cálculos complejos, que no pueden asociarse con ninguna información concreta. Los aspectos dinámicos del sistema se modelan con clases de control
 
- Realización de caso de uso-análisis:** Es una colaboración dentro del modelo de análisis que describe cómo se lleva a cabo y se ejecuta un CU determinado en términos de las clases del análisis y de sus objetos del análisis en interacción.
- Paquete del análisis:** Proporcionan un medio para organizar los artefactos del modelo de análisis en piezas manejables. Un paquete de análisis puede constar de clases de análisis, realizaciones de casos de uso, e incluso otros paquetes. Los paquetes del análisis deberían ser cohesivos y débilmente acoplados. Además, los paquetes del análisis pueden representar una separación de intereses de análisis. Deberían crearse basándose en los requisitos funcionales y en el dominio del problema, y deberían ser reconocibles por las personas con conocimiento del dominio. Probablemente estos paquetes se conviertan en subsistemas a futuro.
 - Paquetes de Servicio:** Son un tipo particular de paquete, proporcionado por el sistema al cliente, para que ofrezca a sus usuarios los casos de uso necesarios para llevar a cabo su negocio. Los paquetes de servicio contienen un conjunto de clases relacionadas funcionalmente. Estos paquetes son indivisibles, reutilizables, y son fundamentales para el diseño y la implementación.
- Descripción de la Arquitectura (Vista del modelo de análisis):** Permite ver la descomposición del modelo de análisis en paquetes de análisis y sus dependencias; las clases fundamentales del análisis y las realizaciones de casos de uso

coordinacion
entre clses de
Interfaz y
Entidad



Flujo de Trabajo del Análisis

1- Análisis de la arquitectura: El propósito de análisis de la arquitectura es esbozar el modelo de análisis y la arquitectura mediante la identificación de paquetes del análisis, clases del análisis evidentes, y requisitos especiales comunes

- a. Identificación de paquetes del análisis: Los paquetes proporcionan un medio para organizar el modelo de análisis en piezas más pequeñas y manejables. Una identificación inicial se hace de manera natural basándonos en los requisitos funcionales y en el dominio de problema, agrupando un cierto número de casos de uso en un paquete concreto, y realizando la funcionalidad correspondiente dentro de dicho paquete.
- b. Identificación de clases de entidad obvias: Basándose en clases del dominio o entidades de negocio
- c. Identificación de requisitos especiales comunes: como gestión de transacciones, persistencia, distribución, concurrencia, seguridad, tolerancia a fallos.

2- Analizar un Caso de Uso: Los objetivos para analizar un caso de uso son:

- ✓ Identificar las clases del análisis cuyos objetos son necesarios para llevar a cabo el flujo de suceso del caso de uso.
- ✓ Distribuir el comportamiento del caso de uso entre objetos del análisis que interactúan.
- ✓ Capturar requisitos especiales sobre la realización del caso de uso.

Para esto, debemos realizar la:

- a. Identificación de clases del análisis: Buscamos clases de entidad, control, e interfaz y esbozamos sus nombres, responsabilidades, atributos, y relaciones
- b. Descripción de interacciones entre objetos del análisis: Para armar un diagrama de colaboración
- c. Captura de requisitos especiales (inherentes al caso de uso que se está tratando)

3- Analizar una Clase: Los objetivos para analizar una clase son:

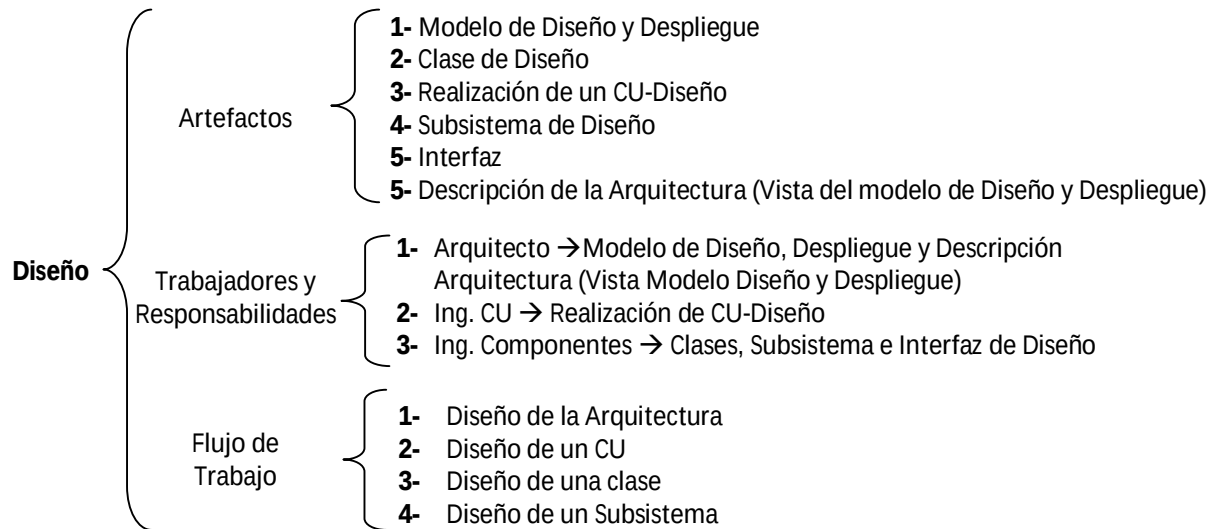
- ✓ Identificar y mantener las responsabilidades de la clase, basadas en su papel en las realizaciones de casos de uso.
- ✓ Identificar atributos y relaciones de la clase.
- ✓ Capturar requisitos especiales sobre la realización de la clase.

Debemos identificar responsabilidades, atributos, asociaciones, agregaciones, generalizaciones y requisitos especiales.

4- Analizar un paquete: Se debe tener en cuenta:

- Garantizar que el paquete sea lo más independiente posible de otros paquetes
- Garantizar que el paquete cumple con sus objetivos
- Describir las dependencias para poder estimar el efecto de cambios futuros

2.3 FLUJO DE TRABAJO DE DISEÑO



Diseño. Actividad mental de disponer las cosas para que al desarrollarse, se alcance la situación deseada. En otras palabras, es la disposición de estructuras temporales que son necesarias para que un algo se desarrolle según nuestros deseos

Disciplina de Diseño: Encuentra una forma de sistema, código o arquitectura, que al momento de su puesta en ejecución da lugar a lo delineado en el análisis, siempre teniendo en foco los requisitos a cumplir.

El diseño es la etapa de un sistema que describe como se implementará el sistema, en un nivel lógico sobre el código real. En el diseño, las decisiones estratégicas y tácticas se toman para resolver los requisitos funcionales y de calidad requeridos de un sistema. Los resultados de esta etapa son representados por los modelos a nivel de diseño, especialmente la vista estática, vista de máquina de estados, y vista de interacción. Una entrada esencial al diseño es el modelo de análisis.

Modelo de Análisis	Modelo de Diseño
Modelo conceptual.	Modelo físico (implementación)
Genérico respecto al diseño (aplicable a varios diseños)	Específico para una implementación
Tres estereotipos: entidad, control, interface.	Cualquier n° de estereotipos físicos.
Menos formal.	Más formal.
Menos caro de desarrollar	Más caro.
Menos capas.	Más capas.
Dinámico (no muy centrado en la secuencia)	Dinámico (muy centrado en la secuencia)
Creado principalmente como trabajo manual	Creado fundamentalmente como "programación visual" en ing.de ida y vuelta.
Puede no mantenerse todo el ciclo de vida.	Debe ser mantenido todo el ciclo de vida.

El diseño es el centro de atención al final de la fase de elaboración y comienzo de las iteraciones de construcción. Esto contribuye a una arquitectura estable y sólida, y a crear un plano del modelo de implementación. El modelo de diseño es muy cercano al de implementación, lo cual es natural para guardar y mantener el modelo de diseño a través del ciclo de vida completo del software.

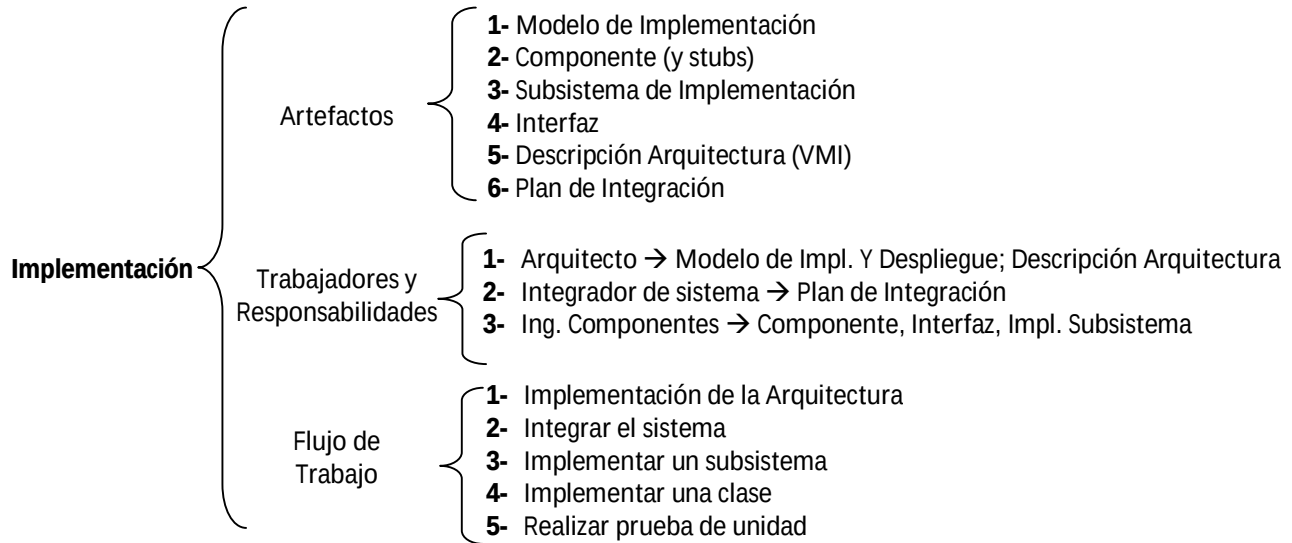
Artefactos del Diseño

- Modelo del Diseño: Es un modelo de objetos que describe la realización física de los casos de uso centrándose en los requisitos funcionales como en los no funcionales. Las abstracciones del modelo de diseño tienen una correspondencia directa con los elementos físicos del ambiente de implementación.
- Clase del Diseño: Es una abstracción sin costuras con una clase o construcción similar en la implementación del sistema. A diferencia de la clase de análisis, se especifica el lenguaje de programación, visibilidad de atributos y operaciones; se implementan las relaciones con atributos; y se pueden usar estereotipos del lenguaje de programación elegido.
- Realización de CU-diseño: Es una colaboración en término de clases de diseño que describe como se realiza un caso de uso específico. Una realización de CU-diseño, tiene una traza directa a la correspondiente realización del caso de uso análisis. Se describe utilizando Diagrama de Clases, de secuencia, y flujo de sucesos-diseño (narración de sucesos)
- Subsistema de diseño: Un subsistema es un paquete de elementos que se tratan como una unidad. Los subsistemas tienen un conjunto de interfaces que describen su relación con el resto del sistema y las circunstancias en que se puede utilizar. Son una forma de organizar los artefactos del modelo de diseño en piezas más manejables. Los subsistemas pueden representar productos software reutilizados, o sistemas heredados encapsulados.
 - o *Subsistemas de Servicio*: Se usan en un nivel inferior de la jerarquía de subsistemas. La identificación de subsistemas de servicio se basa en los paquetes de servicio del modelo de análisis, y normalmente existe una traza uno a uno. Un subsistema de servicio ofrece servicios en término de interfaces y operaciones, y suele dar lugar a un componente ejecutable en la implementación.
- Interfaz: Las interfaces se utilizan para especificar las operaciones que proporcionan las clases y los subsistemas del diseño. Se dice que una clase de diseño o un subsistema de diseño “realizan” o implementan una interfaz. Las interfaces constituyen una forma de separar la especificación de la funcionalidad en términos de operaciones de sus implementaciones en términos de métodos.
- Descripción de la arquitectura (vista del modelo de diseño): Se consideran significativos para la arquitectura los siguientes artefactos del modelo de diseño:
 - o La descomposición del modelo de diseño en subsistemas, sus interfaces, y las dependencias entre ellos.
 - o Clases de diseño fundamentales como clases activas y clases centrales.
 - o Realizaciones de caso de uso-diseño que describan alguna funcionalidad importante y crítica.
- Modelo de despliegue: El modelo de despliegue es un modelo de objetos que describe la distribución física del sistema en términos de cómo se distribuye la funcionalidad entre los nodos de cómputo.
- Descripción de la arquitectura (vista del modelo de despliegue): La descripción de la arquitectura contiene una vista de la arquitectura del modelo de despliegue que muestra sus artefactos relevantes para la arquitectura.

Flujo de Trabajo del Diseño

- 1- Diseño de la arquitectura:** Su objetivo es esbozar los modelos de diseño y despliegue.
 - a. Identificar nodos y configuraciones de red: Bandwidth, protocolos, disponibilidad, etc
 - b. Identificar Subsistemas e interfaces: Constituyen un medio para organizar el modelo de diseño en piezas manejables.
 - c. Id. Clases de diseño significativas para la arquitectura: Por ejemplo, las clases activas.
 - d. Identificar Mecanismos de diseño genéricos: tratan requisitos comunes, Ej: persistencia
- 2- Diseño de un caso de uso:** Comprende las etapas de:
 - a. Identificar clases del diseño participantes
 - b. Descripción de las interacciones entre objetos del diseño
 - c. Identificar subsistemas e interfaces participantes
 - d. Descripción de interacciones entre subsistemas
 - e. Captura de requisitos de implementación del Caso de uso
- 3- Diseño de una clase:** Implica definir para una clase sus operaciones, atributos, relaciones, métodos (que son realizados por las operaciones), ciclo de vida (máquina de estados), dependencias, requisitos relevantes a su implementación, interfaces, etc.
 - a. Esbozar la clase del diseño:
 - i. *Diseñar clases de Interfaz:* depende de la tecnología que se use (VB, Java, etc.)
 - ii. *Diseñar clases de Entidad:* implica aplicar aspectos de persistencia
 - iii. *Diseñar clases de control:* encapsulan secuencias, coordinación de otros objetos, y a veces lógica del negocio. Se deben tener en cuenta: aspectos de distribución en nodos de red, de rendimiento, y de transacción.
 - b. Identificar Operaciones: Se describen utilizando el lenguaje de programación. Notar:
 - i. Responsabilidades de clases de análisis que tienen traza con la clase de diseño
 - ii. Requisitos especiales de la clase de análisis que tienen traza
 - iii. Interfaces que la clase de diseño necesita proporcionar
 - iv. Realizaciones de CU-diseño en las que la clase participa
 - c. Identificar Atributos: Identificamos atributos requeridos por la clase y los describimos utilizando el lenguaje de programación que se usará.
 - d. Identificar Asociaciones y agregaciones: Se debe tener en cuenta la navegabilidad
 - e. Identificar generalizaciones: Las generalizaciones deben implementarse utilizando herencia si el lenguaje lo permite. Si el lenguaje no lo admite debe utilizarse asociación / agregación.
 - f. Describir los métodos: Pueden describirse los algoritmos de los métodos utilizando lenguaje natural, pseudocódigo, o el lenguaje de programación específico. Sin embargo esto suele diferirse hasta la implementación de la clase.
 - g. Describir estados: Mediante diagramas de estado
 - h. Tratar requisitos especiales: aquellos no considerados anteriormente
- 4- Diseño de un subsistema:**
 - a. Mantenimiento de las dependencias entre subsistemas
 - b. Mantenimiento de interfaces proporcionadas por el subsistema
 - c. Mantenimiento de los contenidos de los subsistemas

2.4 FLUJO DE TRABAJO DE IMPLEMENTACIÓN



Disciplina de Implementación: Transforma en código máquina lo diseñado. También asume la redacción de los manuales de usuario y todos los demás artefactos que se definan como parte del sistema en explotación más que del proyecto

En la implementación empezamos con el resultado del diseño e implementamos el sistema en término de componentes, es decir, ficheros de código fuente, scripts, ficheros de código binario, ejecutables, y similares.

La implementación es el centro durante las iteraciones de construcción, aunque también se lleva a cabo trabajo de implementación durante la fase de elaboración, para crear la línea base ejecutable de la arquitectura, y durante la fase de transición para tratar defectos tardíos.

Artefactos de la Implementación

- Modelo de Implementación: Es un modelo de objetos que describe la realización física de los elementos del modelo de diseño.
- Componente: Es el empaquetamiento físico de los elementos de un modelo, como son las clases en el modelo de diseño. Los componentes tienen relaciones de traza con los elementos que implementan. Es normal que un componente implemente varios elementos, por ejemplo varias clases.
 - o Stubs: Un Stub es un componente con una implementación esquelética o de propósito especial que puede ser utilizado para desarrollar o probar otro componente que depende del stub.
- Subsistema de implementación: Proporcionan una forma de organizar los artefactos del modelo de implementación en trozos más manejables. Un subsistema de implementación se manifiesta a través de un mecanismo de empaquetamiento concreto en un entorno de implementación determinado, como un paquete en Java, un proyecto en VB, etc. Los subsistemas de implementación tienen traza 1-1 con los subsistemas de diseño.

- Interfaz: Pueden ser utilizadas para especificar las operaciones implementadas por componentes y subsistemas de implementación.
- Descripción de la arquitectura: Tiene la visión de la arquitectura del modelo de implementación. Los siguientes artefactos son considerados arquitectónicamente significativos:
 - o Descomposición del modelo de implementación en subsistemas, sus interfaces, y dependencias entre ellos.
 - o Componentes clave que siguen la traza de las clases de diseño significativas arquitectónicamente.
- Plan de integración: El sistema se desarrolla incrementalmente a pasos manejables. Cada paso de construcción debe ser sometido a pruebas de integración. Para prepararse ante el fallo de una construcción se lleva un control de versiones para poder volver atrás a una construcción anterior.

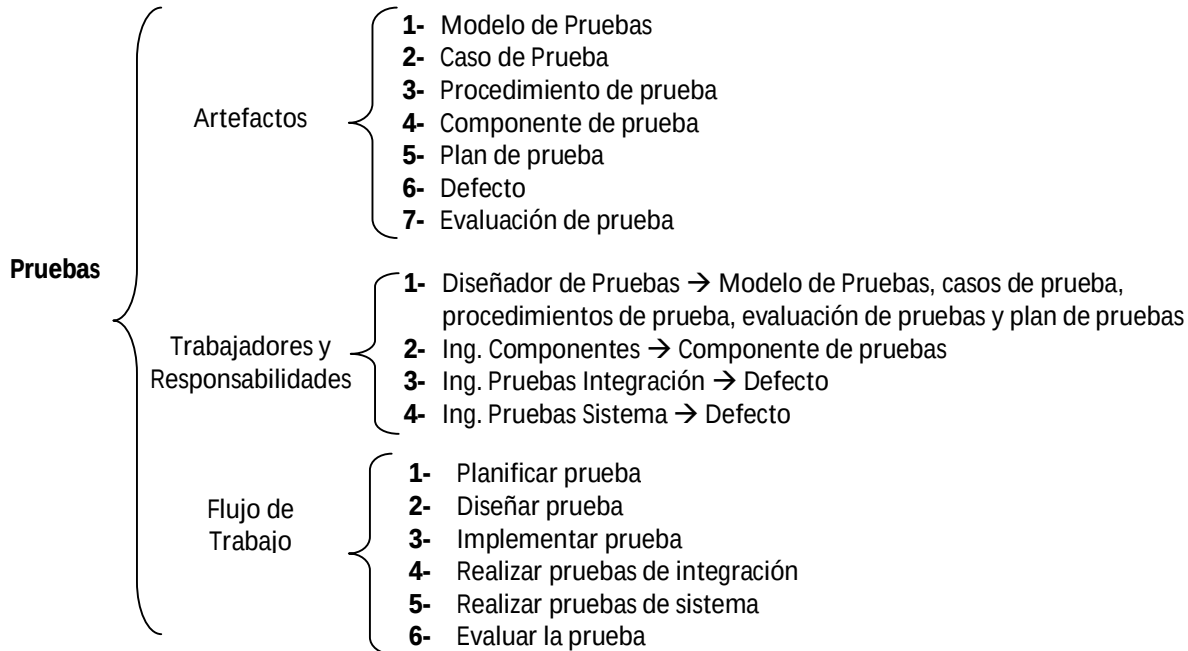
Un plan de integración de construcciones describe la secuencia de construcciones necesarias en una iteración. Un plan de este tipo describe lo siguiente para cada construcción:

- o La funcionalidad que se espera sea implementada en dicha construcción, consiste en una lista de casos de uso a implementar.
- o Las partes del modelo de implementación que están afectadas por la construcción (lista de subsistemas y componentes necesarios para implementar la funcionalidad).

Flujo de Trabajo de la Implementación

- 1- Implementación de la arquitectura:** Tiene como propósito esbozar el modelo de implementación y su arquitectura mediante:
 - a. Identificación de componentes arquitectónicamente significativos tales como componentes ejecutables.
 - b. La asignación de componentes a los nodos en las configuraciones de redes relevantes. Esto se hace asignando un componente ejecutable a cada clase activa.
- 2- Integrar el sistema:** Los objetivos de la integración son:
 - ✓ Crear un plan de integración de construcciones
 - ✓ Integrar cada construcción antes de que sea sometida a pruebas de integración.
- 3- Implementar un subsistema:** Para asegurar que un subsistema cumpla su papel en cada construcción.
- 4- Implementar una clase:** Implementar una clase de diseño en un componente fichero.
- 5- Realizar prueba de unidad:** El propósito de realizar la prueba de unidad es probar los componentes implementados como unidades individuales. Existen dos tipos de prueba:
 - a. Prueba de especificación, o prueba de “caja negra”: verifica el comportamiento de la unidad observable externamente.
 - b. Prueba de estructura o prueba de “caja blanca”: verifica la implementación interna de la unidad.

2.5 FLUJO DE TRABAJO DE PRUEBAS



Disciplina de Pruebas: Define los esquemas de evaluación con los que se determinará si lo desarrollado cumple con los requisitos identificados.

Las pruebas tienen por objetivo encontrar defectos en el software. Hay 2 tipos de pruebas:

- **De caja negra:** los casos de prueba pretenden demostrar que las funciones del software son operativas, que la entrada se acepta de forma adecuada y que se produce una salida correcta. Las pruebas de caja negra se llevan a cabo sobre la interfaz del software, obviando el comportamiento interno y la estructura del programa. Los casos de prueba de la caja negra pretenden demostrar que:
 - o Las funciones del software son operativas
 - o La entrada se acepta de forma correcta
 - o Se produce una salida correcta
 - o La integridad de la información externa se mantiene
- **De caja blanca:** se realiza un examen minucioso de los detalles procedimentales, comprobando los caminos lógicos del programa, comprobando los bucles y condiciones, y examinado el estado del programa en varios puntos. Las pruebas de caja blanca intentan garantizar que:
 - o Se ejecutan al menos una vez todos los caminos independientes de cada módulo
 - o Se utilizan las decisiones en su parte verdadera y en su parte falsa
 - o Se ejecuten todos los bucles en sus límites
 - o Se utilizan todas las estructuras de datos internas

Artefactos de las Pruebas

- Modelo de pruebas: Describe como se prueban los componentes en el modelo de implementación. El modelo de pruebas es una colección de casos de prueba, procedimientos de prueba, y componentes de prueba. **Este modelo implementa una construcción, NO un CU**
- Caso de prueba: Especifica una forma de probar el sistema, incluyendo la entrada o resultado con la que se ha de probar y las condiciones bajo las que ha de probarse. Tiene una relación 1-1 con los escenarios. Un escenario es una secuencia específica de acciones que ilustra un comportamiento. Los escenarios son a los casos de uso, lo que las instancias son a las clases. Los casos de uso describen la funcionalidad aplicable en muchos contextos; los escenarios definen cada contexto y resultado esperado
- Procedimiento de prueba: Dice cómo realizar uno/varios casos de prueba o partes de estos
- Componente de prueba: Automatiza uno o varios procedimientos de prueba o partes de ellos.
- Plan de prueba: Describe las estrategias, recursos y planificación de la prueba. La estrategia incluye definición de tipo de pruebas a realizar por iteración, sus objetivos, nivel de cobertura y código necesario.
- Defecto: Es una anomalía del sistema. Un defecto puede ser utilizado para localizar cualquier cosa que los desarrolladores necesitan registrar como síntoma de un problema.
- Evaluación de prueba: Es una evaluación de los resultados de la prueba.

Flujo de Trabajo de las pruebas

- 1- Planificar prueba**: Consiste en describir una estrategia de la prueba, estimar requisitos para la prueba, recursos humanos y sistemas necesarios; y planificar el esfuerzo de la prueba
- 2- Diseñar la prueba**: Consiste en identificar y describir los casos de prueba para cada construcción, para luego identificar y estructurar los procedimientos de prueba especificando como realizar los casos de prueba.
- 3- Implementar la prueba**: Consiste en automatizar los procedimientos de prueba creando componentes de prueba si esto es posible.
- 4- Realizar pruebas de integración**: Consiste en:
 - a. Realizar las pruebas de integración relevantes ejecutando los procedimientos o componentes de prueba correspondientes.
 - b. Comparar los resultados de las pruebas con los resultados esperados e investigar resultados no esperados.
 - c. Informar defectos a los ingenieros de componentes las fallas encontradas
 - d. Informar los defectos a los diseñadores de pruebas, quienes usarán los defectos para evaluar los resultados de las pruebas.
- 5- Realizar prueba del sistema**: Una vez finalizadas las pruebas de integración se realizan las pruebas de sistema de forma similar.
- 6- Evaluar la prueba**: Se comparan resultados de la prueba con resultados esperados. Para esto se utilizan métricas:
 - a. Compleción de la prueba: indica el porcentaje de casos de prueba que han sido ejecutados y el porcentaje de código que ha sido probado.
 - b. Fiabilidad: Se basa en el análisis de las tendencias den los defectos detectados y en las tendencias en las pruebas que se ejecutan con el resultado esperado.

UNIDAD 3: PATRONES

Los **patrones de diseño** son la base para la búsqueda de soluciones a problemas comunes en el desarrollo de software y otros ámbitos referentes al diseño de interacción o interfaces.

Un patrón de diseño es una solución a un problema de diseño. Para que una solución sea considerada un patrón debe poseer ciertas características. Una de ellas es que debe haber comprobado su **efectividad**, resolviendo problemas similares en ocasiones anteriores. Otra es que debe ser **reusable**, lo que significa que es aplicable a diferentes problemas de diseño en distintas circunstancias.

PATRONES GRASP

GRASP (*General Responsibility Assignment Software Patterns*) son patrones generales de software para asignación de responsabilidades. Aunque se considera que más que patrones propiamente dichos, son una serie de "buenas prácticas" de aplicación recomendable en el diseño de software.

En cuanto a las responsabilidades UML define una responsabilidad como "un contrato u obligación de un clasificador". Las responsabilidades están relacionadas con las obligaciones de un objeto en cuanto a su comportamiento. Básicamente, estas responsabilidades son de los siguientes dos tipos:

- Conocer:
 - o Conocer los datos privados encapsulados.
 - o Conocer los objetos relacionados.
 - o Conocer las cosas que puede derivar o calcular.
- Hacer:
 - o Hacer algo él mismo, como crear un objeto o hacer un cálculo.
 - o Iniciar una acción en otros objetos.
 - o Controlar y coordinar actividades en otros objetos.

Los patrones GRASP son:

EXPERTO EN INFORMACIÓN
Es el principio básico de asignación de responsabilidades. Nos indica que la responsabilidad de la creación de un objeto o la implementación de un método, debe recaer sobre la clase que conoce toda la información necesaria para hacerlo. De este modo obtendremos un diseño con mayor cohesión y así la información se mantiene encapsulada (disminución del acoplamiento)
Problema: ¿Cuál es el principio general para asignar responsabilidades a los objetos?
Solución: Asignar una responsabilidad al experto en información.
Beneficios: Se mantiene el encapsulamiento, los objetos utilizan su propia información para llevar a cabo sus tareas. Se distribuye el comportamiento entre las clases que contienen la información requerida. Son más fáciles de entender y mantener.
Patrones Relacionados: Bajo Acoplamiento, Alta Cohesión

CREADOR
El patrón creador nos ayuda a identificar quién debe ser el responsable de la creación (o instanciación) de nuevos objetos o clases.
Problema: ¿Quién Crea?
Solución: Asignar a la clase B la responsabilidad de crear una instancia de la clase A si: ▪ B Tiene la información necesaria para realizar la creación del objeto A ▪ B Usa directamente las instancias creadas del objeto A ▪ B Almacena o maneja varias instancias de la clase A ▪ B Contiene o agrega la clase A
Beneficios: Una de las consecuencias de usar este patrón es la visibilidad entre la clase creada y la clase creador. Se mantiene bajo acoplamiento, mayor claridad, encapsulación y reutilización
Patrones Relacionados: Bajo Acoplamiento, Todo-Parte (Agregación-Composición)

CONTROLADOR
Sirve como intermediario entre una determinada interfaz y el algoritmo que la implementa, de tal forma que es la que recibe los datos del usuario y la que los envía a las distintas clases según el método llamado. Este patrón sugiere que la lógica de negocios debe estar separada de la capa de presentación. Se recomienda dividir los eventos del sistema en el mayor número de controladores para poder aumentar la cohesión y disminuir el acoplamiento.
Problema: ¿Quién debería encargarse de atender un evento del sistema?
Solución: Asignar la responsabilidad del manejo de un mensaje de los eventos de un sistema a una clase que represente una de las siguientes opciones: • El sistema global, empresa u organización global (controlador de fachada) • Algo en el mundo real que es activo y que puede participar de la tarea (controlador de tareas) • Un manejador artificial de todos los eventos del sistema de un CU (Controlador de Caso de uso)
Beneficios: Aumentar la reutilización de código y a la vez tener un mayor control

ALTA COHESIÓN
La cohesión es una medida de cuán relacionadas y enfocadas están las responsabilidades de una clase. Una alta cohesión caracteriza a las clases con responsabilidades estrechamente relacionadas que no realicen un trabajo enorme. Una clase con baja cohesión hace muchas cosas no afines o un trabajo excesivo. Estas clases no convienen porque son difíciles de reutilizar, comprender, conservar, y además son muy delicadas.
Problema: ¿Cómo mantener la complejidad dentro de límites manejables?
Solución: Asignar una responsabilidad de modo que la cohesión siga siendo alta
Beneficios: • Mejoran la claridad y facilidad con que se entiende el diseño • Se simplifican el mantenimiento y las mejoras en funcionalidad • A menudo se genera un bajo acoplamiento • Permite la reutilización

Agrupar responsabilidades afines en una clase, evitando que haga tareas muy diversas.

BAJO ACOPLAMIENTO
El acoplamiento es una medida de fuerza con que una clase está conectada a otras, con que las conoce y con que recurre a ellas. Una clase con bajo acoplamiento, no depende de muchas otras. Una clase con alto acoplamiento, recurre a muchas otras. Este tipo de clases no es conveniente, ya que disminuye la reutilización, los cambios son difíciles de realizar, y son difíciles de entender.
Problema: ¿Cómo dar soporte a una dependencia escasa y a un aumento de la reutilización?
Solución: Asignar una responsabilidad para mantener bajo acoplamiento
Beneficios: No se afectan por cambios de otros componentes. Son fáciles de entender por separado. Son fáciles de reutilizar

POLIMORFISMO
Se refiere a la capacidad para que varias clases derivadas de una antecesora utilicen un mismo método de forma diferente.
Problema: ¿Cómo manejar las alternativas basadas en el tipo?
Solución: Las responsabilidades del comportamiento se asignarán (mediante operaciones polimórficas) a los tipos en que el comportamiento presenta variantes. <i>No se deben realizar pruebas con el tipo de un objeto ni utilizar lógica condicional para plantear diversas alternativas basadas en el tipo.</i>
Beneficios: Es fácil agregar futuras extensiones que requieran variaciones imprevistas

FABRICACIÓN PURA
Los diseños orientados a objetos se caracterizan por implementar como clases de software las representaciones de conceptos en el dominio de un problema del mundo real; por ejemplo, una clase Venta y Cliente. Pese a ello, se dan muchas situaciones donde el asignar responsabilidades exclusivamente a las clases del dominio origina problemas de mala cohesión o acoplamiento o bien por un escaso potencial de reutilización.
Problema: ¿A quién asignar la responsabilidad cuando uno está desesperado y no quiere violar Alta Cohesión y Bajo acoplamiento?
Solución: Asignar un conjunto cohesivo de responsabilidades a una clase artificial que no representa nada en el dominio del problema: una cosa inventada para dar soporte a una alta cohesión, un bajo acoplamiento y reutilización
Beneficios: Alta cohesión y reusabilidad
Patrones Relacionados: Bajo Acoplamiento, Alta cohesión, Experto

INDIRECCIÓN
El patrón de indirección nos aporta mejorar el bajo acoplamiento entre dos clases asignando la responsabilidad de la mediación entre ellos a un tercer elemento (clase) intermedio
Problema: ¿A quién se asignarán las responsabilidades a fin de evitar el acoplamiento directo? ¿Cómo desacoplar objetos, permitiendo reutilización?
Solución: Se asigna la responsabilidad a un objeto intermedio para que medie entre otros componentes o servicios, y éstos no terminen directamente acoplados.
Beneficios: Bajo Acoplamiento
Patrones Relacionados: Bajo Acoplamiento, Mediador, Fabricación Pura

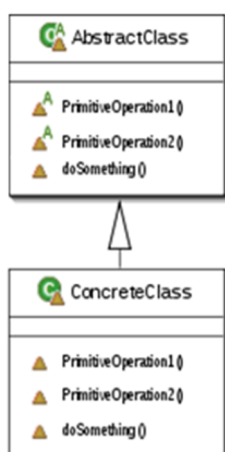
VARIACIONES PROTEGIDAS
Es el principio fundamental de protegerse del cambio, de tal forma que todo lo que preveamos en un análisis previo que es susceptible de modificaciones, lo envolvamos en una interfaz, utilizando el polimorfismo para crear varias implementaciones y posibilitar implementaciones futuras, de manera que quede lo menos ligado posible a nuestro sistema, de forma que cuando se produzca la variación, nos repercuta lo mínimo
Problema: ¿Cómo diseñar objetos, subsistemas y sistemas de manera que las variaciones o inestabilidades en estos elementos no tengan un impacto no deseable en otros elementos?
Solución: Identificar los puntos de variaciones previstas o de inestabilidad asignando responsabilidades para crear una interfaz estable alrededor de ellos
Beneficios: ▪ Se añaden fácilmente las extensiones que se necesitan para nuevas variaciones; ▪ Se pueden introducir nuevas implementaciones sin afectar a los clientes; ▪ Se reduce el acoplamiento; ▪ Puede disminuirse el impacto o coste de los cambios.

PATRONES GoF (Gang of Four)

Son un grupo de patrones de diseño. Los más importantes son:

- Patrones creacionales → Fábrica, Singleton
- Patrones Estructurales → Adaptador, Composite, Decorador, Fachada
- Patrones de Comportamiento → Comando, Observador, Estrategia, Plantilla

Plantilla (Template)

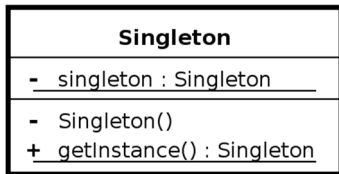


Un método plantilla es un patrón de diseño que define una estructura algorítmica en la súper clase, delegando la implementación a las subclases. Es decir, define una serie de pasos, en donde los pasos serán redefinidos en las subclases.

Se define una estructura de herencia en la cual la superclase sirve de plantilla de los métodos en las subclases, es decir, la superclase define métodos abstractos y las subclases los implementan. Una de las ventajas de este método es que evita la repetición de código, por tanto la aparición de errores.

Este patrón se vuelve de especial utilidad cuando es necesario realizar un algoritmo que sea común para muchas clases, pero con pequeñas variaciones entre una y otras.

Singleton (Instancia única)



El patrón singleton está diseñado para restringir la creación de objetos pertenecientes a una clase o el valor de un tipo a un único objeto.

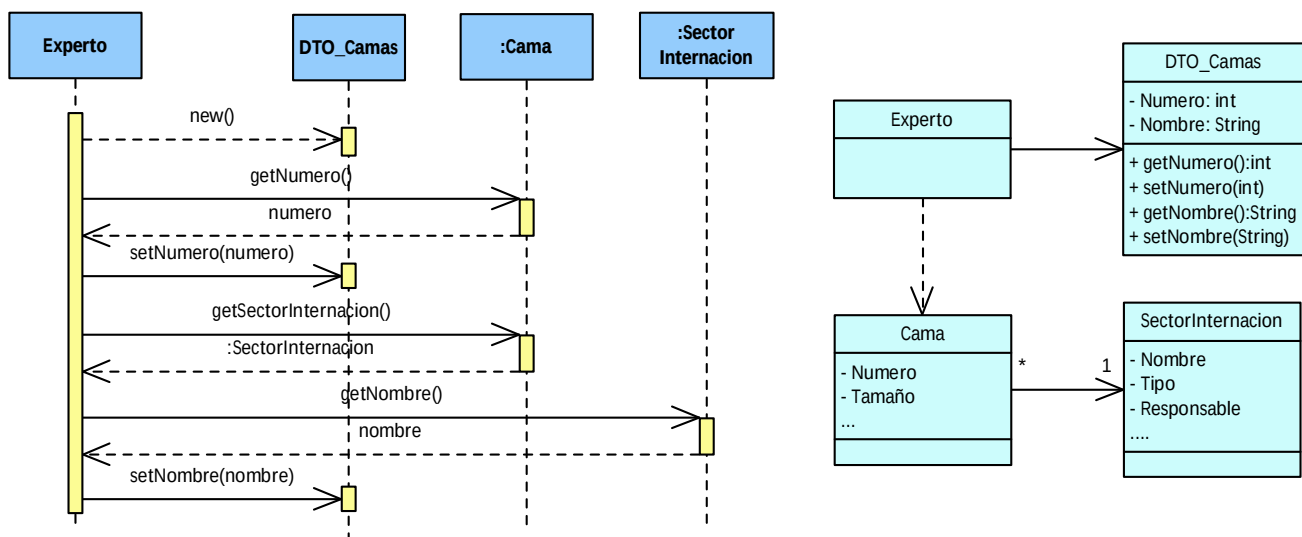
Su intención consiste en garantizar que una clase sólo tenga una instancia y proporcionar un punto de acceso global a ella.

El patrón singleton se implementa creando en nuestra clase un método que crea una instancia del objeto sólo si todavía no existe alguna. Para asegurar que la clase no puede ser instanciada nuevamente se regula el alcance del constructor (con atributos como protegido o privado).

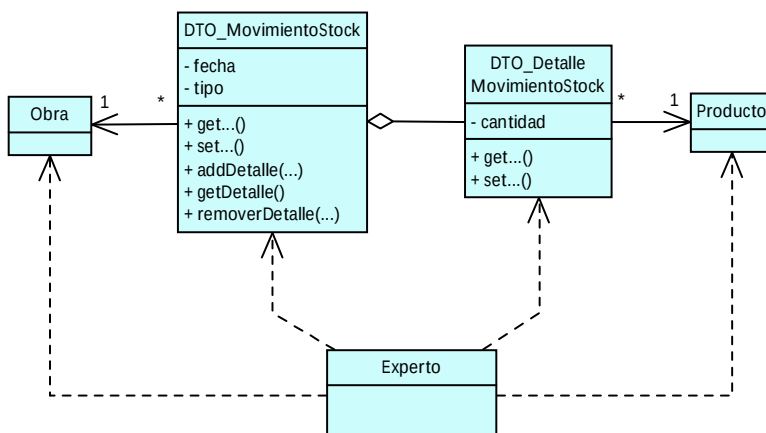
DTO (Data Transfer Object)

Problema: ¿Cómo puedo intercambiar datos entre diferentes capas de una aplicación de manera de mantener un bajo acoplamiento entre ellas?

Solución: Crear pseudo-entidades a medida de las necesidades. Es decir, definir clases con los atributos necesarios para representar los datos que se desean intercambiar entre las diferentes capas.



Ejemplo de Patrón DTO compuesto



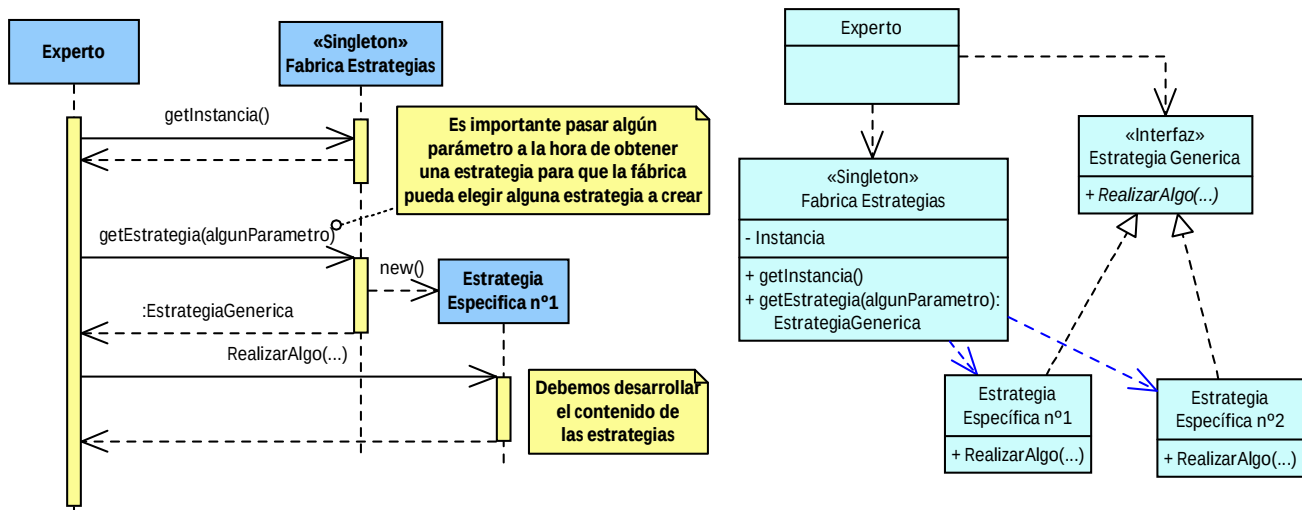
El Patrón DTO se utiliza cuando se necesita pasar información a un sistema externo o a otras partes de nuestro sistema, evitando enviar información necesaria. Como es un objeto creado por nosotros no tiene por lo general lógica alguna con las reglas del negocio

Estrategia (Strategy)

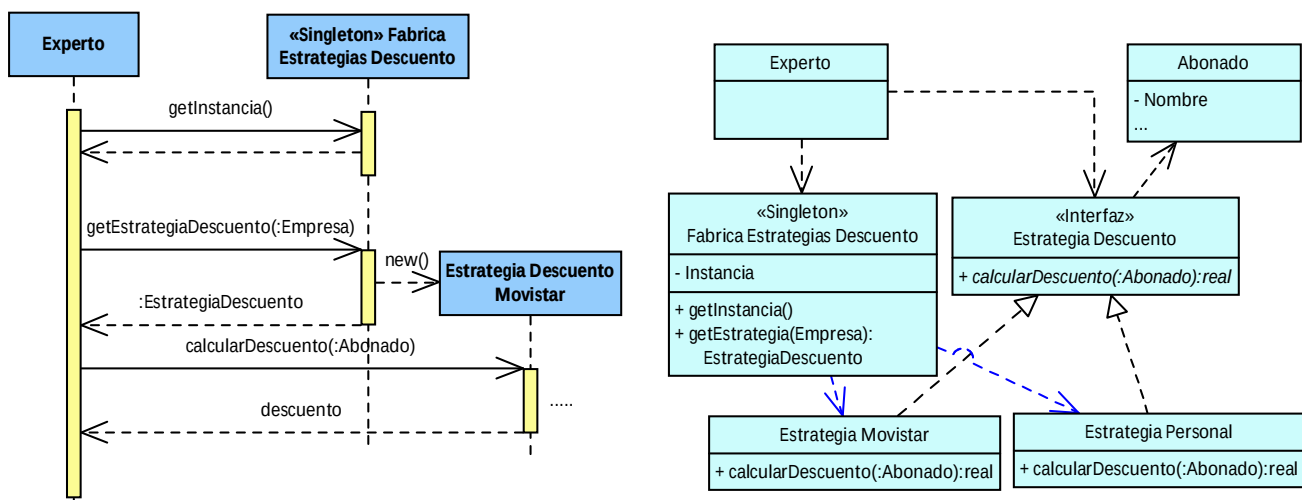
Problema: ¿Cómo diseñar diversos algoritmos o políticas que están relacionadas?

¿Cómo diseñar que estos algoritmos o políticas puedan cambiar?

Solución: Defina cada algoritmo/política/estrategia en una clase independiente con 1 interfaz común



Ejemplo



El patrón estrategia permite mantener un conjunto de algoritmos de los que el objeto cliente puede elegir aquel que le conviene e intercambiarlo según sus necesidades.

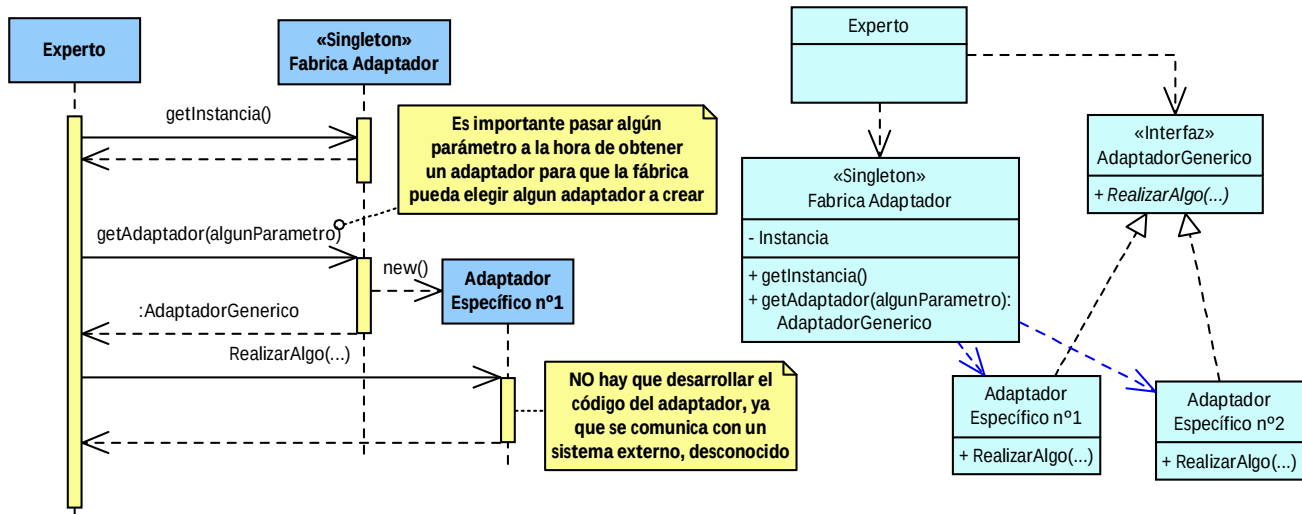
Cualquier programa que ofrezca un servicio o función determinada, que pueda ser realizada de varias maneras, es candidato a utilizar el patrón estrategia. Al utilizar las estrategias, cuando esa parte varía, no debemos modificar el código del experto del C.U. Puede haber cualquier número de estrategias y cualquiera de ellas podrá ser intercambiada por otra en cualquier momento, incluso en tiempo de ejecución. Lo importante de esta clase es que cada una de las estrategias que diseñemos tendrá que sobrescribir un método *RealizarAgo()* y proveer un algoritmo concreto para dicha estrategia.

El contexto es la clase que decide qué estrategia utilizar en cada momento, la decisión se realiza normalmente mediante algún parámetro que le envía el cliente aunque puede ser él mismo el que elija la estrategia más adecuada, por ejemplo, según la fecha.

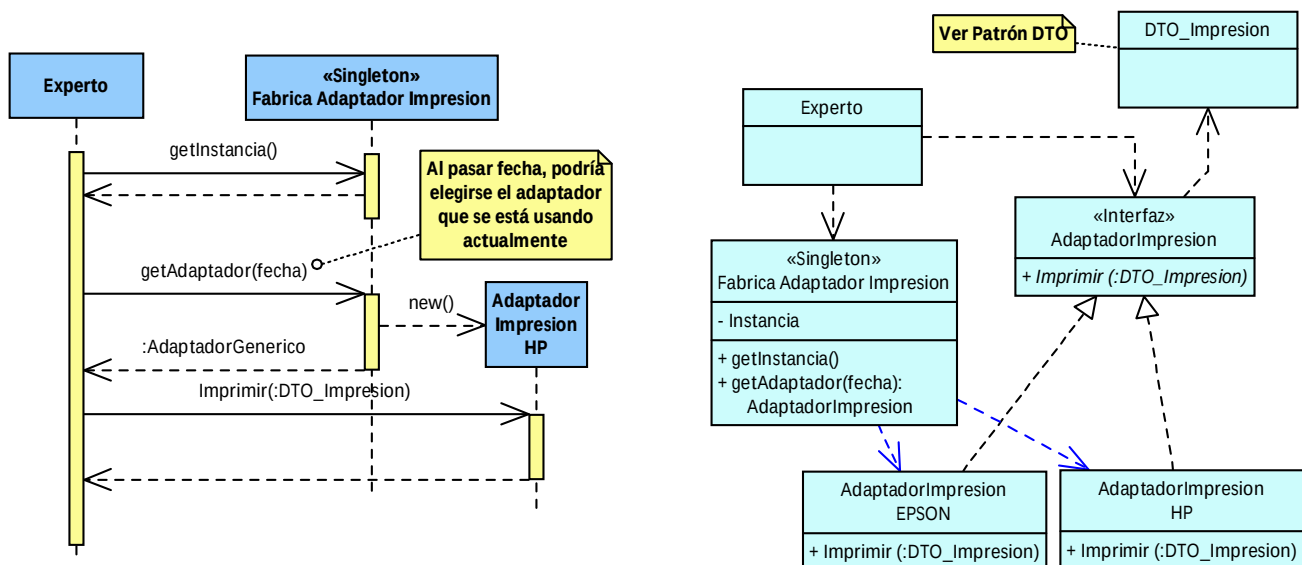
Adaptador (Adapter)

Problema: ¿Cómo resolver interfaces incompatibles, o proporcionar una interfaz estable para componentes parecidos con diferentes interfaces?

Solución: Convierta la interfaz original de un componente en otra interfaz mediante un objeto adaptador intermedio.



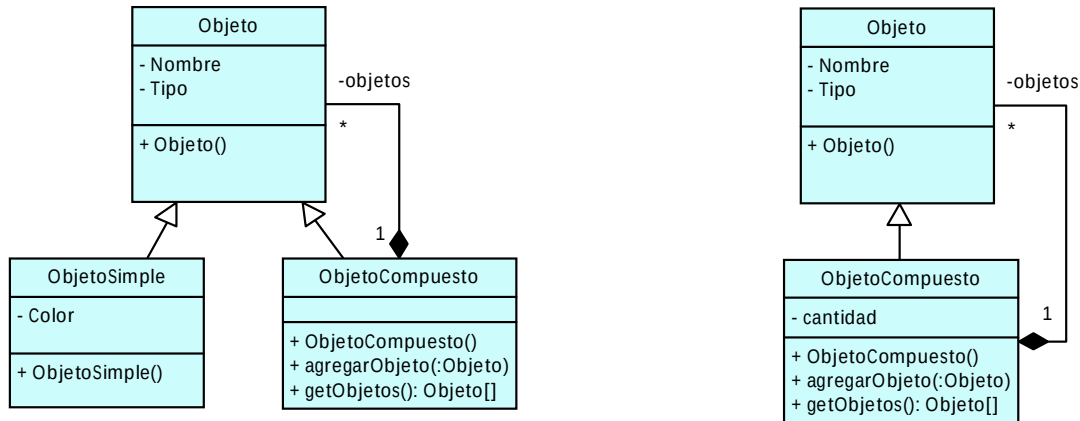
Ejemplo



El Adaptador se utiliza para conectarse con sistemas externos. Suele utilizarse en conjunto con el Patrón DTO, y uno de los usos principales es el de impresión de Factura, Ticket, etc. Cabe aclarar que también puede usarse sin el patrón DTO.

Composite (Objeto Compuesto)

Composite permite tratar un grupo de objetos como una sola pieza, en realidad, como un único objeto. Logra que tanto el objeto compuesto como los objetos componentes sean tratados por igual. Es útil cuando se desea desconocer la diferencia entre el uso de objetos compuestos y objetos componentes



Observador (Observer o Spider)

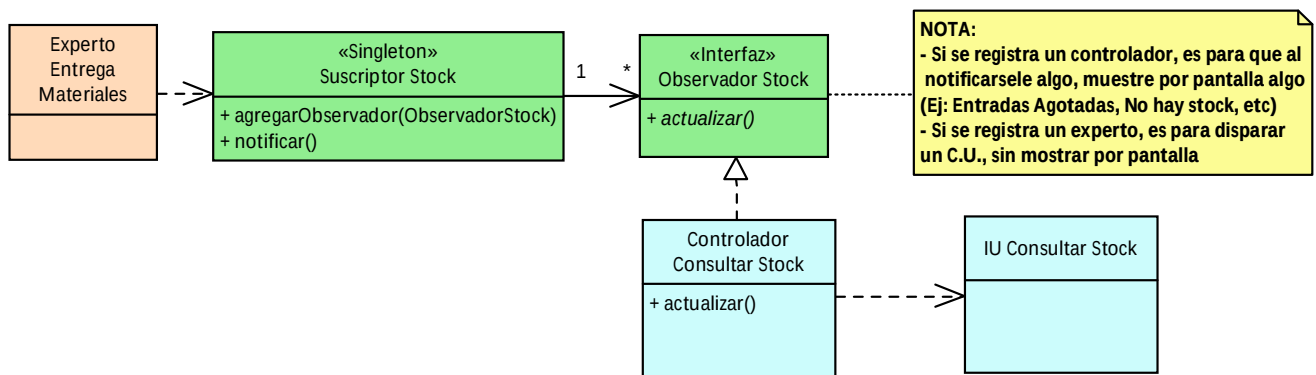
El patrón Observador define una dependencia del tipo uno-a-muchos entre objetos, de manera que cuando uno de los objetos cambia su estado, el observador se encarga de notificar este cambio a todos los otros dependientes.

El objetivo de este patrón es desacoplar la clase de los objetos clientes del objeto, y evitar bucles de actualización (espera activa o polling).

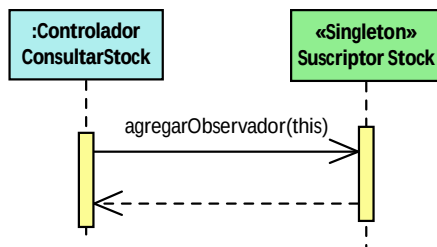
Este patrón también se conoce como el patrón de publicación-inscripción o modelo-patrón. Estos nombres sugieren las ideas básicas del patrón, que son bien sencillas: el objeto de datos, llamémoslo "Sujeto" a partir de ahora, contiene atributos mediante los cuales cualquier objeto observador o vista se puede suscribir a él pasándole una referencia a sí mismo. El Sujeto mantiene así una lista de las referencias a sus observadores.

Los observadores a su vez están obligados a implementar unos métodos determinados mediante los cuales el Sujeto es capaz de notificar a sus observadores "suscritos" los cambios que sufre para que todos ellos tengan la oportunidad de refrescar el contenido representado. De manera que cuando se produce un cambio en el Sujeto, ejecutado, por ejemplo, por alguno de los observadores, el objeto de datos puede recorrer la lista de observadores avisando a cada uno.

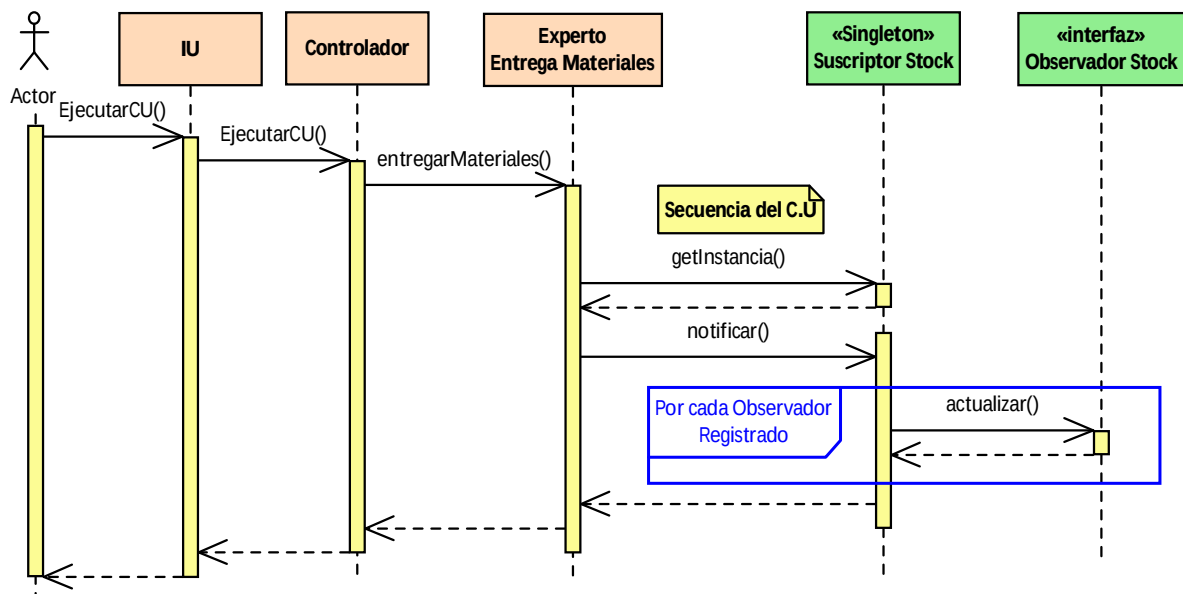
Este patrón suele observarse en los frameworks de interfaces gráficas orientados a objetos, en los que la forma de capturar los eventos es suscribir 'listeners' a los objetos que pueden disparar eventos.



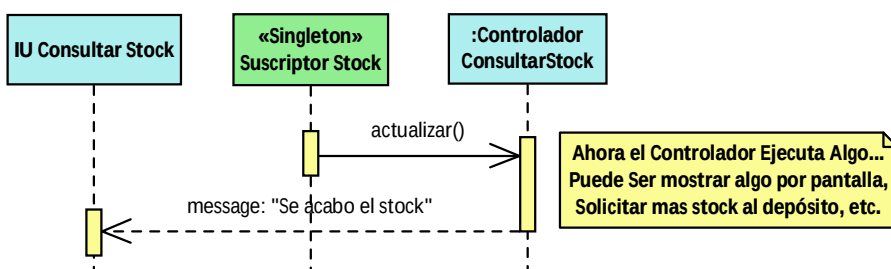
Registro de Observadores



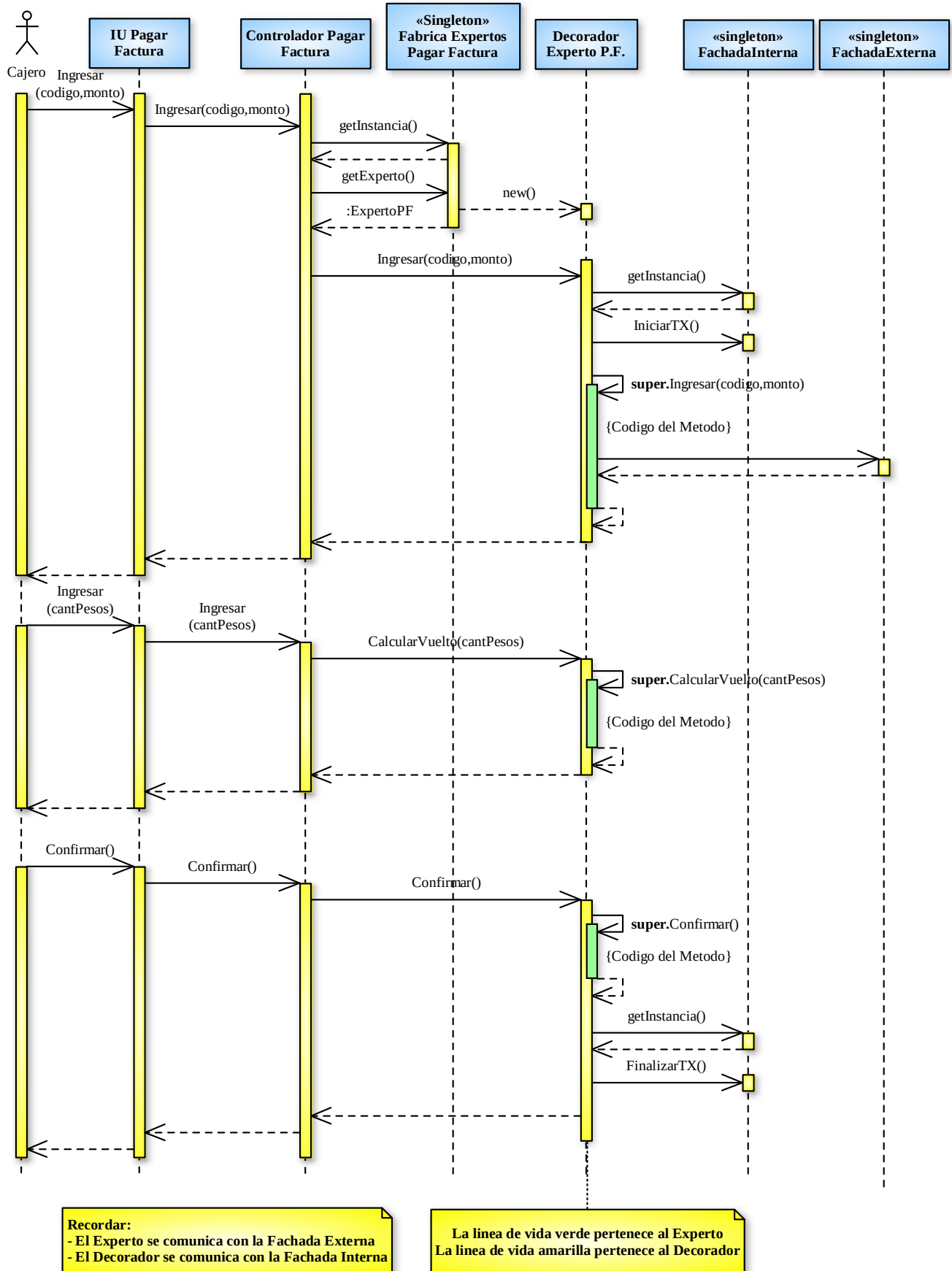
Notificación a Observadores



Observadores reciben la Notificación y hacen algo

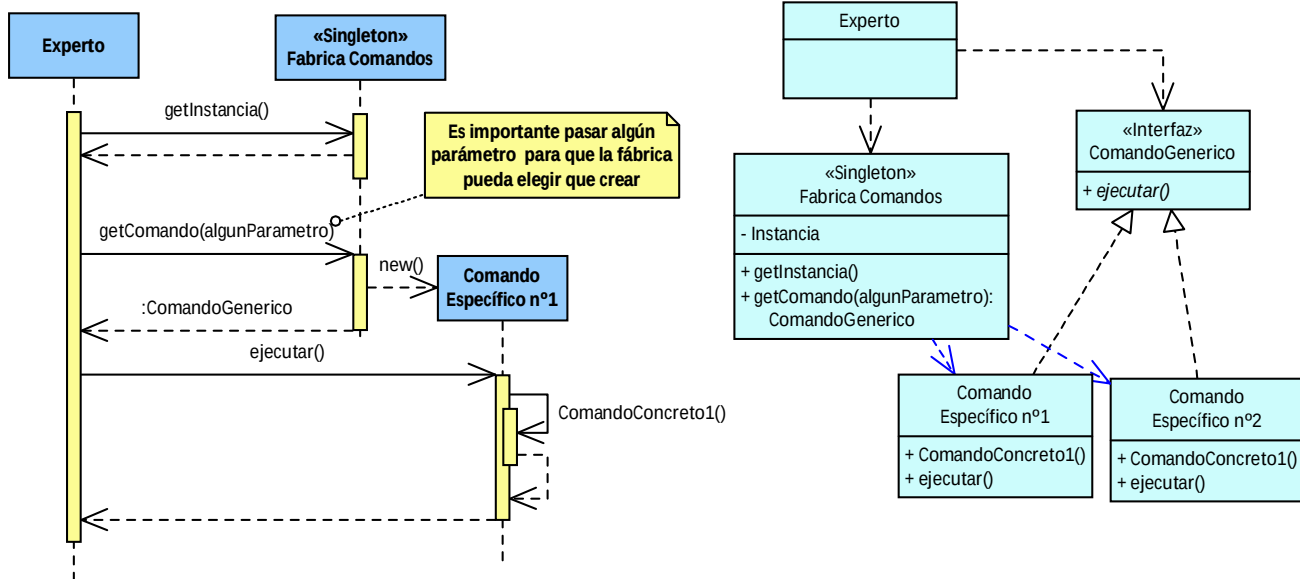


Uso de Experto y Decorador



Comando

Este patrón permite solicitar una operación a un objeto sin conocer realmente el contenido de esta operación, ni el receptor real de la misma. Para ello se encapsula la petición como un objeto, con lo que además se facilita la parametrización de los métodos (por eso se llama siempre con *ejecutar()*)



La habilidad más importante en DOO es la asignación eficiente de responsabilidades a los componentes de software, ya que esto permite:

- Solidez
- Capacidad de mantenimiento
- Reusabilidad de los componentes de software

La segunda habilidad más importante, es la obtención de objetos o abstracciones adecuadas.

¿Qué son el Análisis y el Diseño Orientado a Objetos?

La esencia del AOO y el DOO consiste en situar el dominio de un problema y su solución lógica dentro de la perspectiva de los objetos (cosas, conceptos o entidades).

Durante el Análisis orientado a objetos, se procura identificar y describir los objetos dentro del dominio del problema.

Durante el Diseño orientado a objetos, se procura definir los objetos lógicos del software que finalmente serán implementados en un lenguaje de programación orientado a objetos.

Durante la Construcción o Programación orientada a objetos, se implementan los componentes del diseño

Principios del Diseño Orientado a Objetos

Los principios básicos del diseño orientado a objetos se pueden resumir en:

1. Identificar a los objetos adecuados.
2. Favorecer el bajo acoplamiento.
3. Favorecer la reutilización de código.

Para identificar los objetos adecuados, debemos descomponer el problema en casos de uso, en decir, pensar en “que” y no en “como”.

Una práctica muy habitual es encontrar los objetos a través de los sustantivos y verbos utilizados para definir los casos de uso, los sustantivos se suelen traducir en clases y propiedades mientras que los verbos en métodos.

En el diseño orientado a objetos, los objetos necesitan interactuar y comunicarse, para realizar dicha comunicación, los objetos utilizan su propia interfaz pública, dicha interfaz se compone principalmente de métodos y propiedades.

Para resolver el acoplamiento entre clases, debemos separar la interfaz de la implementación. Si basamos las dependencias entre clases en interfaces, minimizamos el acoplamiento entre clases a un conjunto de funciones.

Para favorecer la reutilización de código, muchas veces aplicamos el concepto de la herencia, sin embargo cuando heredamos de una clase, heredamos un contexto de ejecución y por tanto tenemos visibilidad del estado de la clase heredada, y esto puede llegar a ser problemático por que una clase derivada que utilice el contexto que hereda (los métodos heredados) puede romperse por cambios en la clase padre si dichos cambios alteran el contrato.

Para favorecer la reutilización de código, disponemos de dos opciones:

- Herencia (caja blanca)
- Composición (caja negra)

La composición de objetos, conlleva crear un objeto que cree un “envoltorio” donde alojar una instancia del objeto que deseamos reutilizar y que se referencie a través de un miembro privado,

Cuando creamos un objeto “envoltorio”, estamos favoreciendo la composición sobre la herencia, pero entonces, ¿Cuándo usar herencia?, usaremos herencia cuando:

- Queremos compartir métodos a través de la clase base.
- Queremos aprovecharnos del polimorfismo (métodos abstractos y virtuales).

Con la composición los cambios en el objeto compuesto no afectan al objeto interno, además los cambios del objeto interno, no afectan al objeto contenedor mientras no impliquen modificar la interfaz pública.

Más Principios

- ✓ Responsabilidad Única: Cada clase debe ser responsable de realizar solo una actividad del sistema
- ✓ Clase Abierta y Cerrada: Una clase debe ser abierta a la extensión y cerrada a la modificación
- ✓ Principio de Substitución de Liskov: Cada clase que hereda de otra puede usarse como su padre sin necesidad de conocer las diferencias entre ellas
- ✓ Principio de Inversión de Dependencia: Los módulos de nivel superior no deben depender sino de las abstracciones. Los detalles deben depender a su vez de las abstracciones, no al contrario
- ✓ Principio de Segregación de Interfaces: La implementación de las abstracciones (que contradictorio) debe estar en la medida de lo posible en interfaces y no en clases
- ✓ Principio de Equivalencia de Reúso y Distribución: Solo los componentes que se distribuyen de manera final pueden ser reutilizados, el elemento más importante es entonces el paquete.
- ✓ Principio de Cierre Común: Los componentes que comparten funciones entre ellos o que dependen uno del otro deberían ser colocados juntos
- ✓ Principio de Reúso Común: Si se reutiliza una clase de un paquete entonces se reutilizan todas
- ✓ Principio de Dependencias acíclicas: Los paquetes y sus dependencias no deben formar ciclos entre si
- ✓ Principio de Dependencias Estables: Los paquetes menos estables han de depender de los paquetes más estables
- ✓ Principio de Abstracción Estable: Los paquetes deben ser más abstractos mientras más estables son

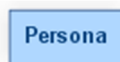
4.2 MODELO DE ENTIDAD – RELACIÓN (MER)

El Modelo de Entidad Relación es un modelo de datos basado en una percepción del mundo real que consiste en un conjunto de objetos básicos llamados entidades y relaciones entre estos objetos, implementándose en forma gráfica a través del Diagrama Entidad Relación.

Las **Entidades** son objetos o cosas del mundo real, del mismo tipo. Tienen atributos, que comprenden valores. Los atributos deben ser:

- No compuestos: Ej: **Domicilio**, tiene número, calle, etc. Debería dividirse en varios atributos
- No multivalente: Es decir, no debe tener permitir distintos tipos de valores
- No nulo
- Determinante (ID, llave o clave primaria): Es un atributo especial único entre todas las instancias, que identifica unívocamente a cada instancia. El ID puede ser:
 - o *Simple*: Formado por un solo atributo. Ej: **Legajo**
 - o *Compuesto*: Formado por 2 o más atributos. Ej: **Tipo y Nro. Documento**

Una entidad se representa con un Rectángulo.



Se denomina **Clave principal** o **primaria** al atributo o conjunto mínimo de atributos (uno o más campos) que permiten identificar en forma única cada instancia de la entidad, es decir, a cada registro de la tabla. Las claves principales se utilizan cuando se necesita hacer referencia a registros específicos de una tabla desde otra tabla. En un principio se puede identificar más de un atributo que cumpla las condiciones para ser clave, los mismos se denominan **Claves candidatas**.

Si la clave primaria se determina mediante un solo atributo de la entidad, entonces se dice que la misma es una **Clave simple**. En caso de estar conformada por más de un atributo, la misma se conoce como **Clave compuesta**.

La **Clave foránea** (también llamada externa o secundaria) es un atributo que es clave primaria en otra entidad con la cual se relaciona

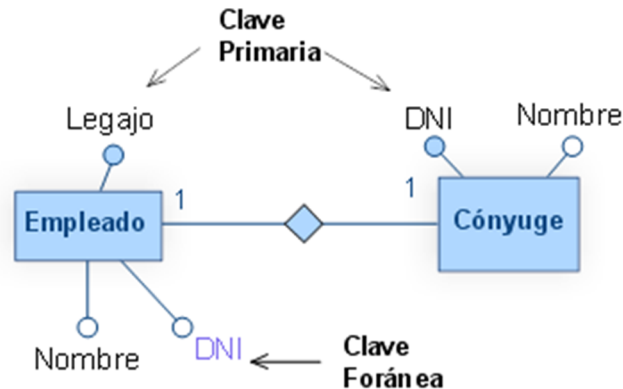
Relación

Es una conexión. Hay una relación cuando al menos una instancia de una entidad se vincula con una instancia en la otra. Las relaciones se representan con un rombo.

EL MER representa una estructura estática del sistema, y hay que tratar de que no varíe (aspecto estructural) es decir, las relaciones deben ser recordadas por el sistema. Puede haber más de una relación entre 2 entidades. Evitamos relaciones n-arias y usamos binarias.

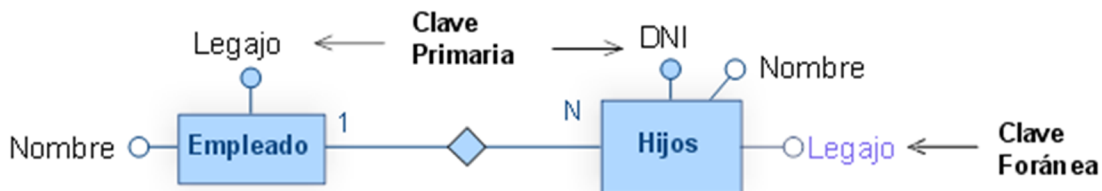
Los tipos de relaciones son:

- 1 a 1: A una instancia de un objeto le corresponde una instancia en el otro.



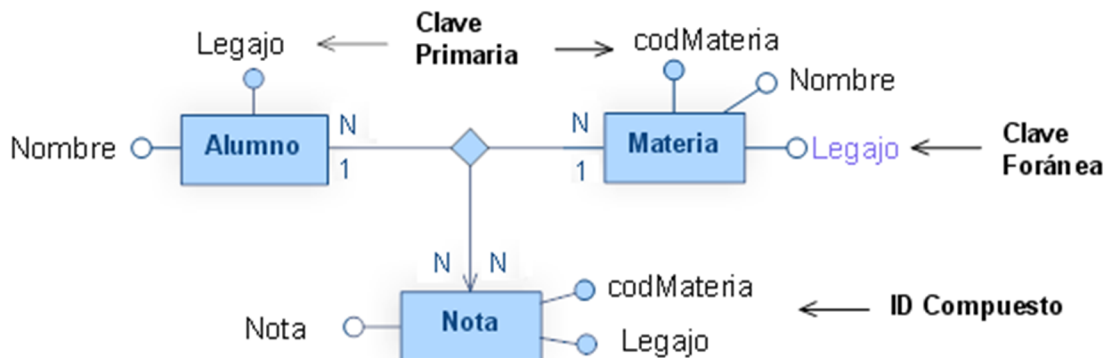
Debe haber al menos 1 atributo en común, en alguna de las 2 entidades, como clave foránea. Para el caso del ejemplo, DNI en Empleado es la clave foránea (referida a Cónyuge)

- 1 a N:



Debe haber al menos 1 atributo en común (clave foránea), en la entidad de cardinalidad N.

- N a N:



Se desprende obligatoriamente una clase asociativa, que tiene como clave primaria los ID's de las 2 relaciones.

Formas Normales

Son principios para hacer óptima una base de datos. Son 9 formas, de las cuales veremos 3.

1. Se define una clave única, no nula, para una tabla
2. Todos los atributos no llave dependen totalmente de la llave compuesta (Solo se aplica a claves compuestas)
3. Los atributos no llave no dependen entre sí

Tomemos un ejemplo para ver la aplicación de las formas normales:

1FN: Se define una clave única, no nula, para una tabla

N° Factura: 12302		Nombre Cliente: Pedro Fuentes		
Fecha: 12/12/03				
Cod. Cliente: A33				
Código Producto	Descripción	Cantidad	Precio Unitario	Subtotal
205	Cuaderno	4	10	40
306	Lápiz	3	1	3
002	Goma	2	0.5	1
....				
Total:				\$ 44

Escribimos los atributos de una super-entidad sin normalizar

NºFactura + fecha + codCliente + nombreCliente + codProd + descripcionProd + cantidad + \$unit + Subtotal + Total

Procedemos a eliminar los atributos calculables, es decir, *Subtotal* y *Total*, quedando:

NºFactura + fecha + codCliente + nombreCliente + codProd + descripcionProd + cantidad + \$unit

Podemos analizar los registros anteriores:

NºFactura	fecha	codCliente	nombreCliente	codProd	descripcionProd	cantidad	\$unit
12302	12/12/03	A33	Pedro Fuentes	205	Cuaderno	4	\$ 10
12302	12/12/03	A33	Pedro Fuentes	306	Lápiz	3	\$ 1
12302	12/12/03	A33	Pedro Fuentes	002	Goma	2	\$ 0.5

Los registros en **rojo** forman un grupo repetitivo, por lo que *NºFactura* por si mismo, no puede ser clave primaria (porque no es única).

Para esto, podemos tomar como clave primaria, a *NºFactura + codProd*, quedando la entidad en 1FN como:

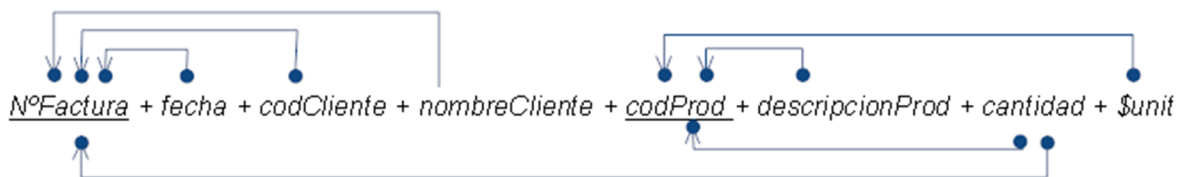
NºFactura + *fecha* + *codCliente* + *nombreCliente* + *codProd* + *descripcionProd* + *cantidad* + *\$unit*

2FN: “Todos los atributos no llave dependen totalmente de la llave compuesta”

Esta forma se aplica SOLO A CLAVES COMPUESTAS

En este ejemplo, nuestra super-entidad en 1FN quedó con una clave compuesta.

Procedemos a analizar que atributo depende de cual de las claves (simbolizamos esto con flechas).



Poe ejemplo, *fecha* depende sólo de *NºFactura* (y no de *codProd*).

En cambio, *cantidad*, depende tanto de *NºFactura* como de *codProd*.

Una vez analizadas las dependencias, las dividimos, quedando la super-entidad en 2FN como:

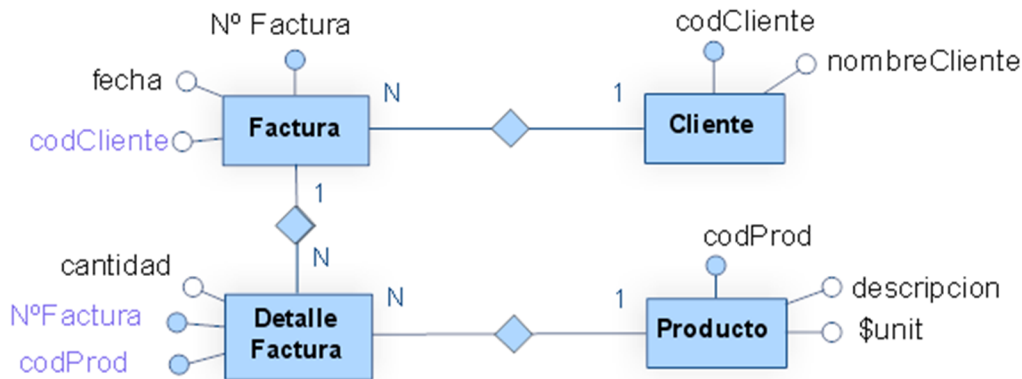
NºFactura + *fecha* + *codCliente* + *nombreCliente*

codProd + *descripcionProd* + *\$unit*

NºFactura + *codProd* + *cantidad* → Como es clave compuesta, la volvemos a revisar por las dudas

3FN: “Los atributos no llave no dependen entre sí”

Analizando la super-entidad en 2FN, podemos ver, por ejemplo, que *nombreCliente* depende de *codCliente*, por lo que los separamos. Además, nombramos las entidades y dibujamos las relaciones, quedando:



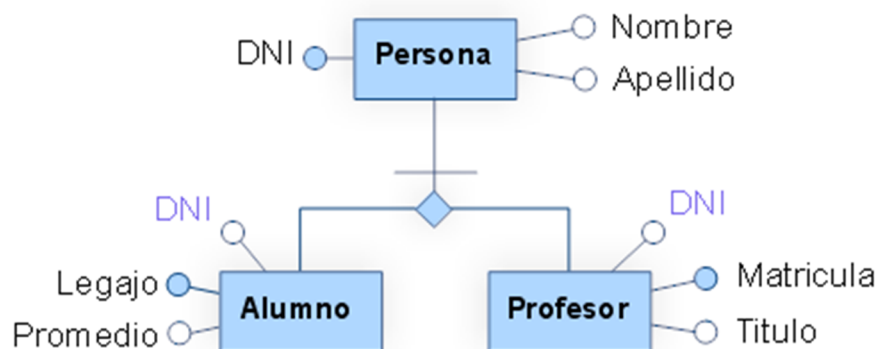
CLIENTE = codCliente + nombreCliente

FACTURA = N°Factura + fecha + codCliente

PRODUCTO = codProd + descripcionProd + \$unit

DETALLE FACTURA = N°Factura + codProd + cantidad

Otros tipos de Relaciones: Herencia



Algunos detalles a tener en cuenta:

- En la super-entidad (Persona, en el ejemplo), van los atributos en común que van a tener las especializaciones (Alumno y Profesor). No se deben repetir estos atributos en las especializaciones, se sobreentiende que están presentes
- Cada sub-entidad tiene una nueva clave primaria, conformada por los atributos que se crean convenientes
- Cada sub-entidad tiene como clave foránea la clave primaria de la super-entidad

4.3 FRAMEWORKS

La palabra inglesa framework define, en términos generales, un conjunto estandarizado de conceptos, prácticas y criterios para enfocar un tipo de problemática particular, que sirve como referencia para enfrentar y resolver nuevos problemas de índole similar.

En el desarrollo de software, un framework es una estructura conceptual y tecnológica de soporte definida, normalmente con artefactos o módulos de software concretos, con base en la cual otro proyecto de software puede ser organizado y desarrollado. Típicamente, puede incluir soporte de programas, bibliotecas y un lenguaje interpretado entre otros programas para ayudar a desarrollar y unir los diferentes componentes de un proyecto.

Representa una arquitectura de software que modela las relaciones generales de las entidades del dominio. Provee una estructura y una metodología de trabajo la cual extiende o utiliza las aplicaciones del dominio.

Nota: Ver Apuntes de:

- *Herencia y Persistencia*
- *Esquema de Persistencia*

UML es un lenguaje de modelado para Visualizar, Especificar, Construir y Documentar los artefactos de un sistema con gran cantidad de software.

Un lenguaje de modelado es un lenguaje cuyo vocabulario y reglas se centran en la representación conceptual y física de un sistema.

El vocabulario y las reglas de UML indican cómo crear y leer modelos bien formados, pero no dicen qué modelos se deben crear ni cuando se deberían crear. Ésta es la tarea del proceso de desarrollo de software.

Normalmente, los proyectos y las organizaciones desarrollan su propio lenguaje, y es difícil comprender lo que está pasando para alguien nuevo o ajeno al grupo. Además, hay cuestiones sobre un sistema que no se pueden entender a menos que se construyan modelos que trasciendan el lenguaje de programación textual. Otro punto a destacar, es que si el desarrollador que escribió el código no dejó documentación escrita sobre los modelos que había en su cabeza, esa información se perderá para siempre, o como mucho, será sólo parcialmente reproducible a partir de la implementación.

Para comprender UML, se necesita adquirir un modelo conceptual del lenguaje, y esto requiere aprender 3 elementos principales:

1- Bloques Básicos de construcción UML: Son 3, elementos, relaciones y diagramas

1. Elementos

a. Estructurales o Clasificadores: *Son las partes estáticas de los modelos*

- Clase: Es una descripción de un conjunto de objetos que comparten los mismos atributos, operaciones, relaciones y semántica
- Interfaz: Es una colección de operaciones que especifican un servicio de una clase o componente. Una interfaz describe el comportamiento visible externo de ese elemento
- Colaboración: Define una interacción y es una sociedad de roles y otros elementos que colaboran para proporcionar un comportamiento cooperativo mayor que la suma de los comportamientos de sus elementos
- Caso de uso
- Clase Activa: Es una clase cuyos objetos tienen uno o más procesos o hilos de ejecución y, por tanto, pueden dar origen a actividades de control.
- Componente: Es una parte modular del diseño del sistema que oculta su implementación tras un conjunto de interfaces externas.
- Artefacto: Es una parte física y reemplazable de un sistema que contiene información física. Representa típicamente el empaquetamiento físico de código o información en tiempo de ejecución.

- Nodo: Es un elemento físico que existe en tiempo de ejecución y representa un recurso computacional, que por lo general dispone de algo de memoria y/o capacidad de procesamiento.

b. De Comportamiento: *Son las partes dinámicas de los modelos*

- Interacción: Es un comportamiento que comprende un conjunto de mensajes intercambiados entre un conjunto de objetos, dentro de un contexto particular, para alcanzar un propósito específico.
- Máquina de Estados: Es un comportamiento que especifica las secuencias de estados por las que pasa un objetos durante su vida en respuesta a eventos. Puede incluir transiciones, eventos, acciones.
- Actividad: Es un comportamiento que especifica la secuencia de pasos que ejecuta un proceso.

c. De Agrupación: *Son las partes organizativas de los modelos UML*

- Paquete: Es puramente conceptual (sólo existe en tiempo de desarrollo).

d. De Anotación: *Son las partes explicativas de los modelos UML*

- Nota: Muestra comentarios

2. Relaciones

- Dependencia: Es una relación semántica entre 2 elementos, en la cual un cambio al elemento independiente puede afectar a la semántica del elemento dependiente
- Asociación: Es una relación estructural entre clases que describe un conjunto de enlaces, los cuales son conexiones entre objetos. Algunos tipos especiales de asociación son la composición y agregación.
- Generalización: Es una relación de especialización, en la cual el elemento especializado (hijo) se basa en la especificación del elemento generalizado (padre). El hijo comparte la estructura y el comportamiento del padre
- Realización: Es una relación semántica entre clasificadores, en donde un clasificador especifica un contrato que otro clasificador garantiza que cumplirá.
- Extensión: Un caso de uso extiende a otro cuando sin alterar a este, se incorpora su funcionalidad como parte integral del primero. Se denota con una relación que apunta del caso extendido al caso base y la conexión se hace o bien al principio del flujo de eventos principal del caso base o en alguno de los puntos de extensión que este haya definido.
- Inclusión: Un caso de uso concreto incluye a un fragmento de caso de uso, cuando como parte de su descripción breve o su flujo de eventos, se hace referencia al texto del fragmento; de forma tal que lo dicho en el fragmento pasa a ser parte de la especificación del caso de uso.

3. Diagramas de...

- Clases: Muestra un conjunto de clases, interfaces y colaboraciones, así como sus relaciones. Abarcan la vista de diseño estática de un sistema
- Casos de Uso: Muestra un conjunto de CU, sus actores y relaciones. Abarcan la vista de casos de uso estática de un sistema.
- Objetos: Muestra un conjunto de objetos y sus relaciones. Representan instantáneas estáticas de instancias de los elementos de un diagrama de clases.
- Componentes: Representa la encapsulación de una clase, juntos con sus interfaces, puertos y estructura interna. Abarcan la vista de implementación estática del diseño de un sistema
- De interacción (Secuencia y Comunicación/Colaboración): Cubren la vista dinámica de un sistema. Muestran el flujo de mensajes entre objetos.
- Estados: Muestra una máquina de estados. Abarca la vista dinámica de un objeto.
- Actividades: Muestra la estructura de un proceso. Abarca la vista dinámica.
- Despliegue: Muestra la configuración de nodos de procesamiento en tiempo de ejecución y los artefactos que residen en ellos. Abarcan la vista de despliegue estática de una arquitectura.
- Paquetes: Muestra la descomposición del propio modelo en unidades organizativas y sus dependencias
- Tiempos: Es un diagrama de interacción que muestra los tiempos reales en el sistema.
- Visión Global de Interacciones: Es un híbrido entre un diagrama de actividades y uno de secuencia.

2- Reglas de Combinación de Bloques

UML tiene reglas sintácticas y semánticas para:

- Nombres: Cómo llamar a los elementos, nombres y diagramas
- Alcance: El contexto que da un significado específico a un nombre
- Visibilidad: Cómo se pueden ver y utilizar esos nombres por otros
- Integridad: Cómo se relacionan apropiada y consistentemente unos elementos con otros
- Ejecución: Qué significa ejecutar o simular un modelo dinámico

3- Mecanismos comunes en UML

UML incorpora 4 mecanismos para simplificar la modelización:

- Especificaciones**: Implica la representación de un concepto mediante una figura. Ej: El óvalo de caso de uso, rectángulo de 3 compartimientos de clase, etc.
- Adornos**: Ej: símbolos +, -, # en atributos de clase
- Divisiones Comunes**: Representar con una misma forma gráfica 2 conceptos
- Mecanismos de Extensibilidad**
 - Estereotipos: Construcción de nuevos bloques a partir de bloques ya existentes
 - Valores Etiquetados: Son detalles agregados
 - Restricciones

ARQUITECTURA

La arquitectura de un sistema puede describirse mediante 5 vistas:

- 1. Vista de Casos de Uso:** Comprende los casos de uso que describen el comportamiento del sistema tal y como es percibido por los usuarios finales, analistas y encargados de las pruebas. Esta vista no especifica realmente la organización de un sistema software.
 - a. Aspectos Estáticos → Diagrama de Casos de Uso
 - b. Aspectos Dinámicos → Diagramas de Interacción, Estados y Actividades

- 2. Vista de Diseño:** Comprende las clases, interfaces y colaboraciones que forman el vocabulario del problema y su solución. Esta vista soporta principalmente los requisitos funcionales del sistema.
 - a. Aspectos Estáticos → Diagramas de Clases y Objetos
 - b. Aspectos Dinámicos → Diagramas de Interacción, Estados y Actividades

- 3. Vista de Interacción:** Muestra el flujo de control entre sus diversas partes. Abarca el rendimiento, escalabilidad y capacidad de procesamiento del sistema.

Los aspectos son los mismos que en la vista de diseño.

- 4. Vista de Implementación:** Comprende los artefactos que se utilizan para ensamblar y poner en producción el sistema físico.
 - a. Aspectos Estáticos → Diagramas de Artefactos
 - b. Aspectos Dinámicos → Diagramas de Interacción, Estados y Actividades

- 5. Vista de Despliegue:** Contiene los nodos que forman la topología hardware sobre la que se ejecuta el sistema. Esta vista se ocupa de la distribución, entrega e instalación de las partes que constituyen el sistema físico.
 - a. Aspectos Estáticos → Diagramas de Despliegue
 - b. Aspectos Dinámicos → Diagramas de Interacción, Estados y Actividades

VISTA ESTÁTICA

La vista estática es la base de UML. Los elementos de la vista estática de un modelo son todo tipo de conceptos encontrados en los sistemas. Esta vista captura la estructura del objeto, e incluye todo lo concerniente a las estructuras de datos tradicionales, así como la organización de las operaciones sobre los datos. Los datos y las operaciones son cuantificados en clases.

La vista estática describe entidades de comportamiento, pero no contiene los detalles de su comportamiento dinámico. Los trata como elementos para ser nombrados, poseídos por las clases e invocados. Hay 2 elementos clave en esta vista:

1. Clasificadores

Un clasificador es un concepto discreto en el modelo, que tiene identidad, estado, comportamiento y relaciones. Algunos clasificadores son las clases, actores, rol, componente, caso de uso, nodo, señal, interfaz, etc.

2. Relaciones

- **Asociación:** Lleva la información entre objetos en un sistema. Sin asociaciones, habrían sólo clases aisladas. Una clase se puede asociar a sí misma, o hacia otra clase. Los extremos de la asociación pueden tener multiplicidad, es decir, cuántas instancias de una clase se pueden relacionar con una instancia de otra. Si una asociación puede tener atributos por sí misma, surge una clase asociativa. Durante el análisis, las asociaciones representan relaciones lógicas entre objetos. La asociación se representa con una línea continua. Hay 2 subtipos:
 - o Agregación: Es una asociación que representa una relación todo-parte. Se muestra adornado con un diamante hueco en el extremo de la trayectoria unida a la clase agregada
 - o Composición: Es una asociación más fuerte, en la cual el compuesto es el responsable único de gestionar sus partes. Se muestra con un diamante relleno adornando el extremo compuesto.
- **Generalización:** Es una relación taxonómica entre una descripción más general (padre) y una descripción más específica (hijo), que se construye sobre ella y la extiende. La descripción más específica es completamente consistente con la más general, y puede contener información adicional. En el caso de clases, se habla de superclase y subclase. Una generalización se dibuja como una flecha que va desde el padre al hijo, con un triángulo hueco en el extremo del padre. La generalización tiene 2 propósitos:
 - o Principio de sustitución (del padre por el hijo, que es más especializado), lo que posibilita el polimorfismo
 - o Permitir la herencia
- **Realización:** Conecta un elemento del modelo con otro que especifica su comportamiento, pero no su estructura o implementación. Se indica con una flecha de línea discontinua con una punta hueca cerrada
- **Dependencia:** Indica que un elemento conoce la existencia de otro. Se denota con una línea punteada y con flecha. En el diagrama de clases sirve para describir la visibilidad global entre atributos.